

Universidad Tecnológica Nacional

Facultad Regional Buenos Aires

Proyecto Final - R6051 - 2024

Aprendizaje por Refuerzo Profundo en un Robot Cuadrúpedo

Informe de Proyecto Final

Piñeiro, Joaquín Mariano
jpineiro@frba.utn.edu.ar

Domínguez Nimo, Alejo
adomnguez@frba.utn.edu.ar

supervisado por
Javier Esarte, Silvio Tapino y Claudia Orlandi

18 de febrero de 2025

1. Agradecimientos

Queremos agradecer a Javier Esarte, parte del "Grupo de Inteligencia Artificial y Robótica" (GIAR), por introducirnos en el aprendizaje por refuerzo, y acompañarnos durante los 2 años que nos tomó realizar este trabajo. Este proyecto no podría haber llegado tan lejos sin su ayuda.

También agradecemos al GIAR por facilitar las impresoras 3D y parte del hardware necesario para el robot; a Lucas Martins por la logística de transporte de los servomotores necesarios que el robot pueda funcionar; a Ariel Sharpe y Matías Marfisi por aportar varias horas de impresión 3D para la fabricación de la estructura; a Lucas Loizaga por prestarnos el hardware para la unidad de medición inercial y a Florencia Counyo por ayudarnos a conseguir los fondos para los servomotores.

2. Resumen

El presente trabajo tiene como objetivo el diseño, fabricación y puesta en marcha de un robot cuadrúpedo con doce grados de libertad, capaz de aprender a caminar de manera autónoma mediante inteligencia artificial, con un algoritmo de aprendizaje por refuerzo profundo. El desarrollo del proyecto abarca la programación de la IA encargada del control de las articulaciones, la simulación de la estructura del robot y los motores, la implementación del firmware con los algoritmos de control para los actuadores, el diseño y la impresión 3D de las piezas que conforman su estructura, así como la realización de pruebas necesarias para alcanzar el objetivo principal: lograr que el robot camine en línea recta de la forma más estable y controlada posible, a una velocidad determinada, manteniendo el cuerpo equilibrado y con la capacidad de ejecutar rotaciones de manera arbitraria.

Primero, se desarrolló el algoritmo de inteligencia artificial. Su funcionamiento fue evaluado en un entorno simulado utilizando un modelo primitivo del robot final. Para ello, se creó un software que facilita la comunicación entre la IA y el simulador, además de proporcionar diversos tipos de datos para analizar su desempeño en tiempo real.

Una vez validado el algoritmo, se procedió al diseño y fabricación de un PCB que interconecta los distintos componentes de hardware, los cuales se detallarán más adelante. Paralelamente, se diseñó e imprimió en 3D la estructura física del robot.

Con el robot ensamblado, se puso a prueba la caminata generada por la IA. Durante este proceso de evaluación, tanto el diseño estructural como los algoritmos y el software involucrados fueron sometidos a un proceso iterativo de mejoras, con el objetivo de optimizar al máximo la estabilidad y el desempeño del robot.

Palabras Claves: Robot Cuadrúpedo, Aprendizaje por Refuerzo Profundo, Soft Actor Critic, Multi-Objetivo, Robótica, Sistemas de Control.

Índice

1 Agradecimientos **2**

2 Resumen **3**

3 Introducción **5**
3.1 Descripción del Problema 5
3.2 Análisis FODA 5
3.3 Matriz de Trazabilidad 6

4 Desarrollo **8**
4.1 Inteligencia Artificial - Multiple Objective Soft Actor Critic (MOSAC) . . 8
4.1.1 ¿Por qué usar aprendizaje por refuerzo? 8
4.1.2 Introducción al aprendizaje por refuerzo 8
4.1.3 Algoritmo Soft Actor-Critic 9
4.1.4 Multi-Objetivo 11
4.1.5 Variables de Estado y Acción 11
4.2 Firmware 12
4.2.1 Acondicionamiento señal potenciómetro. 14
4.2.2 Calibración 19
4.2.3 Control Servomotores 23
4.2.4 Filtro de Kalman 29
4.2.5 Ejecución de Política 38
4.3 PCB 44
4.4 Estructura 47
4.5 Entorno de Simulación - Webots 51
4.5.1 Simulación de estructura del robot 51
4.5.2 Simulación de servomotores del robot 52
4.5.3 Simulación del hardware del robot 62

5 Resultados **63**

6 Conclusiones **67**

3. Introducción

3.1. Descripción del Problema

Los robots cuadrúpedos disponibles en el mercado, como Spot de Boston Dynamics o Go2 de Unitree, logran movimientos biomecánicos avanzados mediante sofisticados y costosos actuadores y sistemas de control. Sin embargo, esta tecnología de alto nivel dificulta su accesibilidad y limita su aplicación en ámbitos donde el presupuesto es una restricción importante. Además, estos robots dependen de hardware especializado, lo que reduce la flexibilidad en cuanto a modificaciones y adaptaciones para diferentes entornos y tareas específicas. En este contexto, surge la necesidad de desarrollar una alternativa más accesible, basada en hardware convencional y sin la necesidad de implementar sistemas de control altamente complejos, pero que aún así permita el aprendizaje y la ejecución de movimientos eficientes.

Para abordar esta problemática, el proyecto propone el desarrollo de un robot cuadrúpedo con doce grados de libertad que emplea inteligencia artificial, específicamente un algoritmo de Aprendizaje por Refuerzo Profundo. A diferencia de otros enfoques, este método no requiere la programación manual de caminatas o sistemas de control avanzados, sino que permite al robot aprender a desplazarse de manera autónoma a través de la experimentación y la optimización de sus movimientos. Además, esta solución no solo facilita la locomoción del robot, sino que también abre la posibilidad de que adquiera nuevos comportamientos adaptados a tareas específicas. La incorporación de sensores adicionales, como un LIDAR, permitiría mejorar sus capacidades de exploración y mapeo autónomo, ampliando su rango de aplicaciones y reforzando la viabilidad de una alternativa más económica y versátil en el campo de la robótica. Esta solución propone un diseño económico del robot con sus partes impresas en 3D y utilizando servomotores como actuadores, en comparación con las alternativas actuales que utilizan materiales como fibra de carbono, aluminio o plástico inyectado y motores brushless. Además robots cuadrúpedos presentes en el mercado actual como el Unitree Go2, utilizan baterías para su funcionamiento de hasta 15000mAh con baterías de 30V, mientras que el diseño que proponemos utiliza una batería de 4000mAh de 7.4V.

3.2. Análisis FODA

Se presenta el análisis de las fortalezas, oportunidades, debilidades y amenazas que se plantearon inicialmente en el proyecto (Fig. 1):

		Oportunidades		Amenazas	
		O1	Funciona aceptablemente en primera prueba. Disponemos de mas tiempo para mejoras	A1	Se rompe una pieza plástica en alguna prueba.
		O2	Consumo menor al esperado	A2	Requiere el uso de una fuente de alimentación de mucha potencia
				A3	Se rompe uno o más servomotores en alguna prueba.
				A4	La simulacion no coincide con el modelo real.
				A5	El torque de los servos no es suficiente para que se pueda mover el robot.
Debilidades		**Estrategias de Adaptación (DO)**		**Estrategias de Supervivencia (DA)**	
D1	Inicialmente cableado.	D1,O1 y O2	Uso de baterías como fuente de alimentación.	A3	Se compra uno o varios servos iguales a los usados.
D2	Desgaste continuo de servomotores por torques producidos en eje del servo (en lugar de en el motor interno de continua)	D2,O1	Si el desgaste hace que empeore demasiado el funcionamiento de los servomotores, se cambia el actuador.	A5	Conseguir financiación lo más rapido posible, y comprar servos con mayor torque. Evaluar cambiar el actuador.
				A2	Uso de múltiples fuentes en paralelo para aumentar la corriente máxima disponible.
Fortalezas		**Estrategias Ofensiva (FO)**		**Estrategias Defensiva (FA)**	
F1	Posibilidad de impresión 3D para mejorar o reparar robot	F2,O1	Posibilidad de trabajar en mejoras al control del robot (velocidad, etc) y correcciones menores para caminata más fluida	F1,A1	Cambio en el diseño 3D del robot para estructura más robusta.
F2	Posibilidad de reentrenar con el mismo algortimo frente a cualquier modificacion en la estructura fisica o agregado de objetivo	F2,O1	Mejorar el aprendizaje para que tenga en cuenta el consumo energético.	F2,A4	Mejora de simulación que contemple delays. Diseño de una política robusta que contemple diversas discrepancias entre la simulacion y realidad.

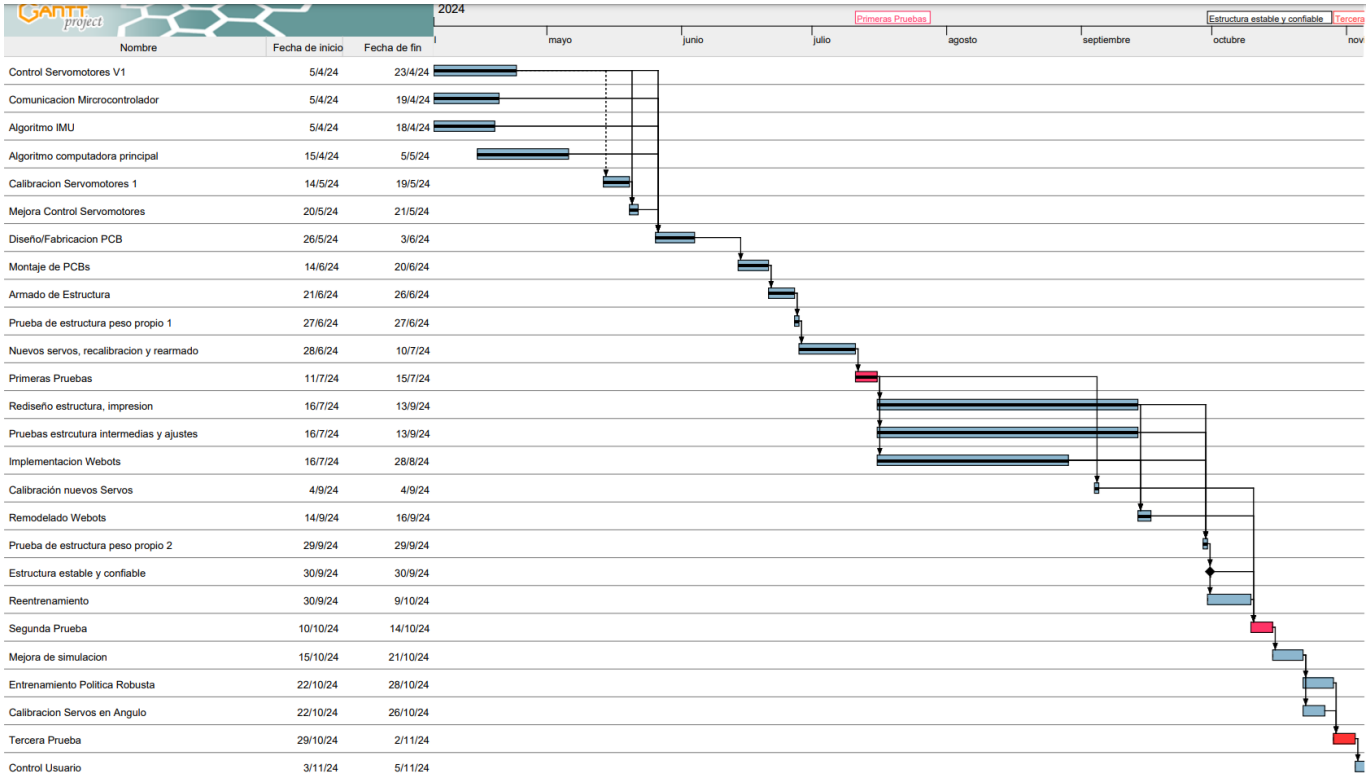
Figura 1: Análisis FODA del proyecto.

3.3. Matriz de Trazabilidad

Se presenta la matriz de trazabilidad (Fig. 2) y los diagramas de Gantt (Fig. 3 y 4) propuestos para el proyecto :

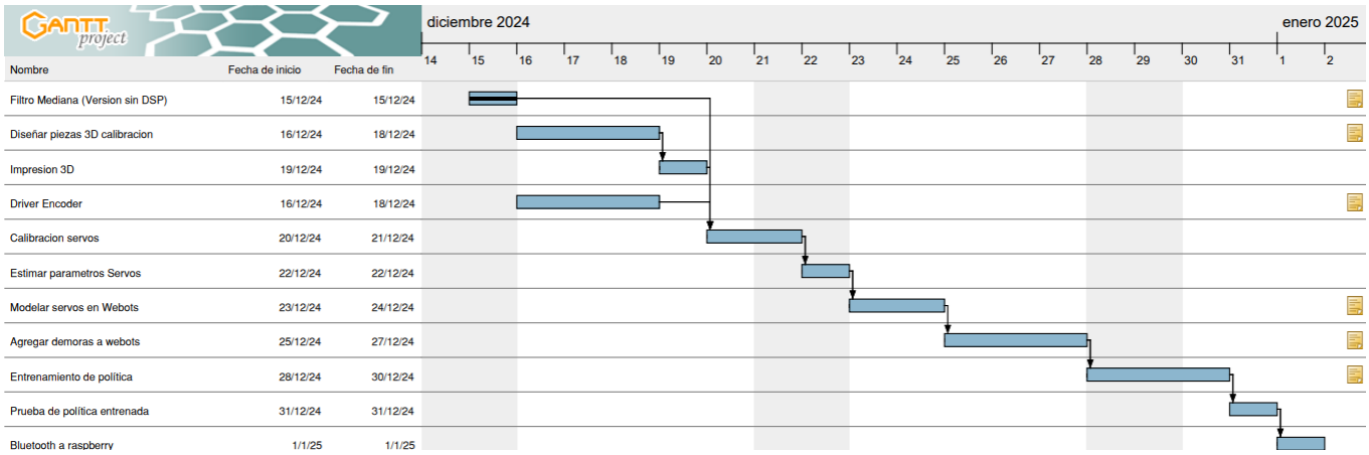
ID	Riesgo	Descripción	Solución	Impacto	Probabilidad	Nivel de Riesgo
1	Discrepancias simulación y realidad.	La simulación no representa fielmente a la realidad y se dificulta la transferencia de lo entrenado al robot físico.	Mejora de la simulación teniendo en cuenta delays, pesos. Entrenamiento de política más robusta aleatorizando parámetros físicos.	Alto	80%-90%	**Alto**
2	Torque insuficiente en los servos.	Los servos no pueden suministrar suficiente potencia mecánica a la estructura como para comportarse adecuadamente dinámicamente.	Reemplazo por servos de mayor potencia mecánica y/o optimización estructura.	Alto	80%-90%	**Alto**
4	Rotura pieza plástica	Se rompe alguna parte del 3D.	Rediseño y reimpresión 3D relativamente rápida.	Bajo	70%-80%	**Medio**
5	Consumo demasiado elevado.	El robot requiere de mucha potencia para funcionar, y no alcanza con la fuente actualmente siendo usada.	Utilización de una batería	Bajo	10%-20%	**Bajo**
5	Rotura servo motores	Los servomotores se dañan ya sea mecánicamente, o eléctricamente.	Usar servomotores de repuesto.	Medio	20%-30%	**Bajo**

Figura 2: Matriz de Trazabilidad.



Nombre	Fecha de inicio	Fecha de fin
Control Servomotores V1	5/4/24	23/4/24
Comunicacion Mircrocontrolador	5/4/24	19/4/24
Algoritmo IMU	5/4/24	18/4/24
Algoritmo computadora principal	15/4/24	5/5/24
Calibracion Servomotores 1	14/5/24	19/5/24
Mejora Control Servomotores	20/5/24	21/5/24
Diseño/Fabricacion PCB	26/5/24	3/6/24
Montaje de PCBs	14/6/24	20/6/24
Armado de Estructura	21/6/24	26/6/24
Prueba de estructura peso propio 1	27/6/24	27/6/24
Nuevos servos, recalibracion y rearmado	28/6/24	10/7/24
Primeras Pruebas	11/7/24	15/7/24
Rediseño estructura, impresion	16/7/24	13/9/24
Pruebas estrcutura intermedias y ajustes	16/7/24	13/9/24
Implementacion Webots	16/7/24	28/8/24
Calibración nuevos Servos	4/9/24	4/9/24
Remodelado Webots	14/9/24	16/9/24
Prueba de estructura peso propio 2	29/9/24	29/9/24
Estructura estable y confiable	30/9/24	30/9/24
Reentrenamiento	30/9/24	9/10/24
Segunda Prueba	10/10/24	14/10/24
Mejora de simulacion	15/10/24	21/10/24
Entrenamiento Politica Robusta	22/10/24	28/10/24
Calibracion Servos en Angulo	22/10/24	26/10/24
Tercera Prueba	29/10/24	2/11/24
Control Usuario	3/11/24	5/11/24

Figura 3: Diagrama de Gantt de la primera parte del proyecto.



Nombre	Fecha de inicio	Fecha de fin
Filtro Mediana (Version sin DSP)	15/12/24	15/12/24
Diseñar piezas 3D calibracion	16/12/24	18/12/24
Impresion 3D	19/12/24	19/12/24
Driver Encoder	16/12/24	18/12/24
Calibracion servos	20/12/24	21/12/24
Estimar parametros Servos	22/12/24	22/12/24
Modelar servos en Webots	23/12/24	24/12/24
Agregar demoras a webots	25/12/24	27/12/24
Entrenamiento de política	28/12/24	30/12/24
Prueba de política entrenada	31/12/24	31/12/24
Bluetooth a raspberry	1/1/25	1/1/25

Figura 4: Diagrama de Gantt de la segunda parte del proyecto.

4. Desarrollo

4.1. Inteligencia Artificial - Multiple Objective Soft Actor Critic (MOSAC)

En este apartado se explica y justifica el uso de la IA utilizada en el robot, con una breve introducción al Aprendizaje por Refuerzo Profundo. El desarrollo de este algoritmo forma parte de un trabajo anexado a este proyecto que se presentó en el ***"2024 IEEE Biennial Congress of Argentina (ARGENCON)"*** [1].

4.1.1. ¿Por qué usar aprendizaje por refuerzo?

Actualmente existen muchos métodos para generar la marcha de un cuadrúpedo. Estos métodos implican la complejidad de diseñar movimientos coordinados de un sistema multiarticulado para asegurar estabilidad y locomoción ágil. Por esta razón, los métodos que requieren un modelo matemático preciso del robot exigen conocer principios de biomecánica y un ajuste fino mediante prueba y error para lograr una solución a medida. En contraste, algunos algoritmos de aprendizaje por refuerzo (RL, por sus siglas en inglés) [2] son capaces de generar locomoción con menor tiempo de sintonización de parámetros y sin un modelo matemático del robot o del entorno. Además, el RL permite al usuario adaptar la marcha para lograr diferentes objetivos mediante el diseño de diversas funciones de recompensa, y permite aplicar técnicas para hacer el controlador más robusto, como la aleatorización de los parámetros dinámicos [3].

4.1.2. Introducción al aprendizaje por refuerzo

La caminata de un cuadrúpedo, al igual que muchos otros problemas en RL, puede describirse como un Proceso de Decisión de Markov (MDP, por sus siglas en inglés), que modela la interacción de un agente dentro de un entorno. En este caso uno pensaría que el agente es el robot y el entorno es el suelo sobre el que camina. Sin embargo, el entorno se puede definir como todo aquello sobre lo que el agente no tiene control. Esto quiere decir que la forma en la que se mueven los motores, el tiempo de respuesta de la electrónica, los sensores y el ruido eléctrico que presentan, las partes mecánicas que forman las articulaciones y la distribución del peso del robot, son todas características propias del entorno y no del agente. Por ende, el agente solo representa el algoritmo que selecciona las acciones que luego debe realizar el robot.

La interacción entre el agente y el entorno se modela en pasos de tiempo discretos: en cada paso de tiempo, al agente se le da un estado, es decir, una representación del entorno (cómo se encuentra el robot en ese instante), y elige una acción a realizar basada en dicho estado. Luego de realizar la acción, el entorno entrega el nuevo estado y una recompensa numérica que informa qué tan buena fue la acción tomada en base al objetivo que se desea que aprenda el agente.

Formalmente, un MDP se suele representar como una tupla de cuatro elementos $(\mathcal{S}, \mathcal{A}, \rho, R)$: $\mathcal{S}$ y $\mathcal{A}$ son espacios continuos de estado y acción; ρ es la Función de

Transición y representa la densidad de probabilidad de que un agente en el estado $s_t \in \mathcal{S}$, ejecutando una acción $a_t \in \mathcal{A}$, transicione al nuevo estado $s_{t+1} \in \mathcal{S}$; R es la función utilizada para calcular la recompensa dada por el entorno basada en el estado, la acción tomada y el siguiente estado $r_t = R(s_t, a_t, s_{t+1})$ donde $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$. Esta última función es muy importante porque, tal como se mencionó anteriormente, es la que indica qué se desea que aprenda a realizar al agente (sea caminar, saltar, subir escaleras, entre otras acciones) en base a qué movimientos se recompensan y cuales son penalizados (recompensa numérica negativa).

En este contexto, el mecanismo por el cual el agente elige una acción a realizar en base al estado actual se denomina **"política"**. Una "función de política" por ende se encarga de mapear estados con sus respectivas acciones elegidas $\pi : \mathcal{S} \to \mathcal{A}$.

Para evaluar las decisiones del agente en relación al objetivo establecido (la calidad de la política) en cada instante t, no se evalúa la recompensa obtenida en ese momento, sino que se utiliza la suma esperada de las recompensas a lo largo del tiempo a partir de dicho instante, denominada **"retorno"**:

$$G(t) = \sum_{t=0}^{\infty} E[\gamma^t R(s_t, a_t, s_{t+1})]$$

Donde $\gamma \in [0; 1]$ es el factor de descuento y representa cuánto impactan las recompensas futuras en el valor del retorno. **El objetivo del agente es lograr en cada paso el mayor retorno posible, teniendo en cuenta no solo la recompensa inmediata, sino también las que podrían obtenerse a futuro**. Esto es muy importante dado que obliga al agente a planificar a mediano e incluso largo plazo (dependiendo del valor de γ).

4.1.3. Algoritmo Soft Actor-Critic

El problema del cuadrúpedo involucra espacios de estado y acción continuos y de alta dimensionalidad (son vectores con muchos elementos), por lo que es una tarea que requiere mucha exploración del entorno teniendo en cuenta las infinitas variantes de estados posibles. Por esta razón se emplea un algoritmo llamado **"Soft Actor-Critic"** (SAC), el cual está diseñado para estos espacios continuos, y se basa en "Aprendizaje por Refuerzo de Máxima Entropía". Esto significa que no solo maximiza la suma esperada de recompensas a lo largo del tiempo, sino que también intenta maximizar la entropía de la política. Esto implica que la función de la política debe ser estocástica, y el mapeo de estados a acciones debe seguir una determinada distribución de probabilidad condicional $\Pi(a_t|s_t)$. Para lograrlo, se hace que la parte determinística de la política no genere directamente la acción directa a tomar sino los parámetros de una distribución de probabilidad Gaussiana μ_a y σ_a, permitiendo una muestra aleatoria de la acción final de $\Pi(a_t|s_t) = \frac{1}{\sigma_a\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{a_t-\mu_a}{\sigma_a}\right)^2\right)$.

Por lo tanto, el objetivo ahora es maximizar:

$$\sum_{t=0}^{\infty} \left[\gamma^t R(s_t, a_t, s_{t+1}) + \alpha H\left(\Pi(\cdot|s_t)\right)\right]$$

Donde H es la entropía de la política π:

$$H\left(\Pi(\cdot|s_t)\right) = \mathbb{E}\left[-\ln\left(\Pi(\cdot|s_t)\right)\right]$$

El factor α es un parámetro de temperatura que ajusta el peso relativo de la entropía en la función de recompensa. La entropía es una medida de la aleatoriedad de una distribución de probabilidad, por lo tanto, una política con mayor entropía indica más incertidumbre en qué acción podría tomarse. Este objetivo modificado fomenta el mantenimiento de un cierto nivel de aleatoriedad en la política, haciendo que el agente explore más ampliamente durante el entrenamiento y reduciendo el riesgo de converger a una política óptima local. Este método evita el uso de parámetros adicionales para equilibrar la exploración y la explotación (un desafío importante en el ámbito de aprendizaje por refuerzo), y supera a algoritmos previos como DDPG, TD3 y PPO en benchmarks complejos [4].

Además de solucionar la exploración y explotación, el algoritmo emplea una arquitectura actor-crítico (de aquí el nombre), ideal para dominios de acción y espacio continuos. El actor es la política, que toma una observación del estado del entorno s_t y produce una acción continua a_t. El crítico es una función que estima qué tan buenas serán las acciones a tomar la política (actor) a partir de cierto estado. Esta función se denomina **"función de valor de estado"**, y devuelve el retorno esperado, estimando de antemano qué acciones tomará la política y qué estados futuros resultarán consecuentemente. Matemáticamente se trata del valor esperado del retorno, siguiendo la distribución de probabilidad de la política y del entorno:

$$V^{\pi}(s_t) = \mathbb{E}_{a_t \sim \Pi(\cdot|s_t), s_{t+1} \sim \rho}\left[G(t)\right]$$
$$V^{\pi}(s_t) = \mathbb{E}_{a_t \sim \Pi(\cdot|s_t), s_{t+1} \sim \rho}\left[\sum_{t=0}^{\infty} E[\gamma^t R(s_t, a_t, s_{t+1})]\right]$$

Esto se puede reescribir haciendo uso de otra función como herramienta adicional:

$$V(s_t) = \mathbb{E}_{a_t \sim \pi(\cdot|s_t)}[Q(s_t, a_t) - \alpha H\left(\Pi(\cdot|s_t)\right)]$$

$Q(s_t, a_t)$ es la "función de valor de estado-acción", también llamada simplemente **"función Q"**. Devuelve la suma esperada de recompensas tomando la acción a_t en el estado s_t y siguiendo posteriormente la política π para las acciones de los siguientes pasos. El algoritmo SAC propuesto por Haarnoja et al. [4] utiliza esta función directamente en lugar de la función $V(s_t)$, dado que es más sencilla de implementar.

Finalmente, tanto la política (actor), como la función Q (crítico) pueden modelarse como redes neuronales, teniendo en cuenta que la red de la política generará para cada acción un valor medio y un desvío estándar que deben utilizarse para generar valores aleatorios con distribución gaussiana que siga dichos parámetros. La red de la política se puede optimizar usando como función de pérdida la red de la función Q negado $-Q(s_t, \pi(s_t))$, y la red de la función Q se optimiza utilizando como base las ecuaciones de Bellman, que aprovechan una propiedad de autoconsistencia que poseen las funciones basadas en el retorno $G(t)$. Para mayor detalle sobre la implementación nos remitimos al trabajo anexado a este proyecto, publicado en ARGENCON 2024 [1].

4.1.4. Multi-Objetivo

Un problema significativo que surge al diseñar la función de recompensa es cómo hacer que el agente logre múltiples objetivos, que en muchos casos son conflictivos. En la caminata de un cuadrúpedo, por ejemplo, un objetivo podría ser alcanzar una cierta velocidad hacia adelante, y otro podría ser inclinar lo menos posible el cuerpo. Este último no es el objetivo principal, sino una restricción, y está en conflicto permanente con el primero: para alcanzar mayores velocidades, es inevitable que haya mayor inclinación del cuerpo.

Por esta razón, se implementó un enfoque multi-objetivo basado en SAC (MOSAC) inspirado en el trabajo de Schaul et al. [5]. Se introduce un vector de "preferencias" $p = [p_1, \ldots, p_n] \in \mathcal{P}$, donde cada elemento es el peso de una recompensa parcial.

Tanto la política como la función Q ahora también tienen en cuenta el vector de preferencias:

$$Q(s_t, a_t, p)$$
$$\pi(s_t, p)$$

Esto significa que el agente no solo tiene que aprender a alcanzar cada uno de los objetivos como antes, sino también a inferir cómo el valor de p dado como entrada afecta el retorno (con la función Q) y actuar de acuerdo con ese vector de preferencias particular para que el retorno sea lo más alto posible (con la política).

4.1.5. Variables de Estado y Acción

Las variables que el entorno le proveerá al agente para que la política pueda tomar decisiones se pueden observar en la tabla 1, y fueron elegidos siguiendo las recomendaciones de Jie Tan [3]. Para normalizar los ángulos de las articulaciones se define un valor máximo y mínimo para cada una, que varía según el tipo de articulación, y luego con la siguiente expresión se mapea el valor máximo a 1 y el mínimo a -1:

$$\bar{\alpha_i} = \left(\alpha_i - \frac{\alpha_{max} + \alpha_{min}}{2} \right) \cdot \frac{2}{\alpha_{max} - \alpha_{min}}$$

Por otro lado, el vector de acción, que se toma a la salida de la política, se compone de los ángulos normalizados de las 12 articulaciones. Para desnormalizarlos se hace la operación inversa a la que figura en la expresión anterior.

Variable	Nombre	Formato
ψ	Rotación target	$\frac{\text{rad}}{\pi}$
θ	Inclinación en Pitch Cuerpo	$\frac{\text{rad}}{\pi}$
ϕ	Inclinación en Roll Cuerpo	$\frac{\text{rad}}{\pi}$
$\dot{\theta}$	Velocidad angular en Pitch Cuerpo	$\frac{\text{rad}}{s\pi}$
$\dot{\phi}$	Velocidad angular en Roll Cuerpo	$\frac{\text{rad}}{s\cdot\pi}$
$\bar{\alpha_i}$	Posición Angular Articulación N°i (0 a 11)	Normalizado de -1 a 1

Tabla 1: Elementos del vector de estados.

4.2. Firmware

El firmware está enfocado principalmente en el control de los actuadores que manipulan las articulaciones del robot, los cuales son **servomotores** controlados por una señal PWM. La principal característica para seleccionar este tipo de actuadores es el torque máximo que pueden ejercer de forma continua, la resolución angular y el ángulo máximo que pueden alcanzar.

Se realizaron diferentes pruebas de esfuerzo mecánico con servomotores comercialmente disponibles en Argentina de la serie FUTABA S3003 y MG996R, con torque máximo de 4,1kgf$\cdot cm$ y 11kgf$\cdot cm$ respectivamente, pero resultaron insuficientes para soportar las cargas estimadas a las que estaría sometido el robot (principalmente su propio peso). Por este motivo, se optó por comprar fuera del país 12 servomotores de 35kgf$\cdot cm$ de 270° que operan con un voltaje de alimentación de 8.4V máximo (Fig. 5), de la marca "DS Servo".

Estos se controlan con una señal PWM cuya frecuencia puede ser de entre 50Hz y 330Hz, y su ancho de pulso varía entre 0.5 ms y 2.5 ms para cubrir un rango de 0 a 270°. Se especifica un parámetro denominado *"dead bandwidth"* de $2\mu s$, el cual es la mínima variación en el ancho del pulso que debe haber para que el servomotor comience a moverse, lo cual significa una resolución de 0.27°.

Otro parámetro importante es la corriente máxima que soporta el servomotor cuando su rotor está bloqueado, la cual es de 3.2A. En base a este valor, se seleccionó la batería que alimenta todo el robot.



Figura 5: Servomotor DS3235.

Es importante mencionar que estos servomotores no disponen de una señal de feedback, para saber en qué ángulo se encuentran. Su uso normal implica averiguar de antemano qué ancho de pulso de la señal de control PWM representa cada ángulo, y suponer que al aplicar cierta señal el servomotor se encuentra en la posición correspondiente. Este enfoque no es suficiente para nuestra aplicación: por un lado es importante saber cómo varía la posición angular en el tiempo al cambiar el ángulo target, es decir, los valores intermedios hasta llegar al ángulo que corresponde a la señal PWM; por otro lado, también puede suceder debido a la carga a la que se someten los actuadores, que el ángulo final no corresponda con el ángulo observado en vacío. Por estas razones es importante tener información sobre el ángulo del servomotor en todo momento, y así implementar un sistema de control que corrija cualquier desviación y sepa cuándo las articulaciones llegaron a la posición de destino.

Para obtener una señal relacionada a la posición angular, se aprovecha el control interno de los servomotores, el cual dispone de un potenciómetro como feedback para controlar el motor de continua. El eje del potenciómetro gira solidariamente con el eje del servomotor, por lo que la tensión en el punto medio es proporcional al ángulo del eje. Entonces, se soldó al punto medio del potenciómetro del servomotor, un cable que provee la posición angular del rotor en Volts.

Para poder controlar los servomotores y a su vez medir los ángulos de cada uno, se necesita un microcontrolador que tenga al menos 12 canales de timer capaces de generar señales PWM, 12 canales de ADC para medir la tensión de los potenciómetros, y que sea lo suficientemente rápido como para realizar un acondicionamiento de las señales y ejecutar el control de las articulaciones sin demora considerable. Por estas razones, se eligió utilizar la placa NUCLEO-F412ZG [6] que tiene un microcontrolador STM32F412 Cortex M4, que dispone de un ADC de 12 bits con 16 canales, 20 canales de timer capaces de generar señales PWM, con timers de sobra para usar en el sistema de control.

4.2.1. Acondicionamiento señal potenciómetro.

El inconveniente principal de medir la señal del potenciómetro es la elevada cantidad de ruido eléctrico que posee la señal, debido a la fricción del potenciómetro y sobre todo por los picos de corriente que consume el motor de continua cuando el servo ejerce el máximo torque (al cambiar el ángulo target), cuyo campo magnético variable induce picos de tensión en la señal del potenciómetro (Fig. 6). Es por esto que es necesario filtrar la señal antes de ser utilizada.



Figura 6: Ruido de la señal del potenciómetro, medido con un DSO en modo Peak Detect.

Inicialmente se utilizó un filtro de media móvil exponencial (EWMA por sus siglas en inglés) para reducir el ruido de la señal:

$$y[n] = x[n] \cdot \alpha + y[n-1] \cdot (1 - \alpha)$$

Se trata de un filtro pasabajos, cuyo parámetro α permite variar la frecuencia de corte del filtrado: al reducir α, se reduce la frecuencia de corte y por ende se elimina en mayor medida el ruido eléctrico. Sin embargo, como los picos producidos por la corriente del motor son muy elevados, para filtrarlos completamente se debe reducir en gran medida la frecuencia de corte, deformando los flancos de la señal original e introduciendo una demora inaceptable.

Por esta razón, se decidió emplear un filtro de mediana. Es parecido al filtro de media móvil, en cuanto a tener una ventana de N muestras de la señal, que se desplaza y obtiene

para cada posición de la ventana un único valor de salida del filtro. La diferencia es que en lugar de obtener la media de los N valores se obtiene la mediana, que corresponde al valor central de la ventana luego de ordenar los datos de menor a mayor (si la ventana es par, se toma el promedio de los dos valores centrales). Se trata de un filtro alineal que tiene la capacidad de eliminar picos de ruido sin comprometer los flancos de la señal original. Para no tener que ordenar los valores de la ventana cada vez que se desplaza, se utiliza un algoritmo detallado por S. Herzog [7] que hace uso de una lista doblemente enlazada.

```
spMedianFilter spCreateMedianFilter(unsigned int size)
{
    spMedianFilter pFilter = NULL;
    // Check if the requested size is valid
    if ((size == 0) || (size == ~0)) { return NULL; }
    // Allocate memory for the structure + flexible array member
    pFilter = (spMedianFilter)malloc(sizeof(struct spMedianFilter) +
    ↪  sizeof(spFilterElement) + size * sizeof(spFilterElement));
    if (pFilter == NULL) { return NULL; }
    // Initialize the structure
    pFilter->size = size;       // Buffer size
    pFilter->rIdx = 1;          // Mark the first element as the next to be replaced
    pFilter->mIdx = 0;          // Median index
    pFilter->state = 0;         // Cleared "full buffer" flag
    pFilter->pData[0].raw = 0;  // Initial median
    // Initialize each element as unused (circular reference in both links)
    size++;
    while (size > 0)
    {
        size--;
        pFilter->pData[size].pIdx = 0;
        pFilter->pData[size].nIdx = 0;
    }
    // Return the pointer to the filter
    return pFilter;
}

void spDestroyMedianFilter(spMedianFilter* ppFilter)
{
    // Release the allocated memory and return
    if ((ppFilter != NULL) && (*ppFilter != NULL))
    {
        free(*ppFilter);
        *ppFilter = NULL;
    }
    return;
}

SP_DATATYPE spMedianFilterMedian(spMedianFilter pFilter)
{
    return pFilter->pData[0].raw;
}
```

```
42
43  SP_DATATYPE spMedianFilterInsert(spMedianFilter pFilter, SP_DATATYPE value)
44  {
45      unsigned int size;
46      unsigned int idx = pFilter->mIdx;
47      const unsigned int rIdx = pFilter->rIdx;
48      spFilterElement* const pData = pFilter->pData;
49
50      // Check if the next element is free (first link leads to itself)
51      if (pFilter->state == 0)
52      { // Free element
53          // Find the position in which the new element should be inserted
54          if (value < pData[pFilter->mIdx].raw)
55          { // The new value is less than the median
56              // Move backwards until the list element is less than the new element
57              for (idx = pData[idx].pIdx; (idx != 0) && (value < pData[idx].raw); idx =
                ↪  pData[idx].pIdx);
58              // Link the new element to its new adjacent elements
59              pData[rIdx].pIdx = idx;
60              pData[rIdx].nIdx = pData[idx].nIdx;
61              pData[pData[rIdx].nIdx].pIdx = rIdx;
62              pData[idx].nIdx = rIdx;
63              // Decrement the median index if there will be an even number of elements
64              if ((!(rIdx & 1)) || (rIdx == 1)) { pFilter->mIdx =
                ↪  pData[pFilter->mIdx].pIdx; }
65          }
66          else
67          { // The new value is greater than or equal to the median
68              // Move forward until the list element is greater than or equal to the new
                ↪  element
69              for (idx = pData[idx].nIdx; (idx != 0) && (value >= pData[idx].raw); idx =
                ↪  pData[idx].nIdx);
70              // Link the new element to its new adjacent elements
71              pData[rIdx].pIdx = pData[idx].pIdx;
72              pData[rIdx].nIdx = idx;
73              pData[pData[rIdx].pIdx].nIdx = rIdx;
74              pData[idx].pIdx = rIdx;
75              // Increment the median index if there will be an odd number of elements
76              if (rIdx & 1) { pFilter->mIdx = pData[pFilter->mIdx].nIdx; }
77          }
78
79          // The number of elements matches the replacement index
80          size = rIdx;
81      }
82      else
83      { // Already in use, overwrite
84          // Find the position in which the new element should be inserted
85          if (value < pData[pFilter->mIdx].raw)
86          { // The new value is less than the old median
87              // Move backwards until the list element is less than the new element
88              for (idx = pData[idx].pIdx; (idx != 0) && (value < pData[idx].raw); idx =
                ↪  pData[idx].pIdx);
89              // Temporarily link the new preceeding element, and move the median index
                ↪  backward if the replaced element used to be after the median or was the
                ↪  median
```

```
90              pData[pData[idx].nIdx].pIdx = rIdx;
91              if ((pData[rIdx].raw > pData[pFilter->mIdx].raw) || (rIdx ==
                ↪  pFilter->mIdx)) { pFilter->mIdx = pData[pFilter->mIdx].pIdx; }
92              pData[pData[idx].nIdx].pIdx = idx;
93              // Unlink the replaced element from its adjacent elements
94              pData[pData[rIdx].pIdx].nIdx = pData[rIdx].nIdx;
95              pData[pData[rIdx].nIdx].pIdx = pData[rIdx].pIdx;
96              // Link the new element to its new adjacent elements
97              pData[rIdx].pIdx = pData[pData[idx].nIdx].pIdx;
98              pData[rIdx].nIdx = pData[idx].nIdx;
99              pData[pData[rIdx].nIdx].pIdx = rIdx;
100             pData[pData[rIdx].pIdx].nIdx = rIdx;
101         }
102         else
103         { // The new value is greater than or equal to the median
104             // Move forward until the list element is greater than or equal to the new
                ↪  element
105             for (idx = pData[idx].nIdx; (idx != 0) && (value >= pData[idx].raw); idx =
                ↪  pData[idx].nIdx);
106             // Temporarily link the new following element, and move the median index
                ↪  forward if the replaced element used to be before the median (older
                ↪  values equal to the median are placed before it)
107             pData[pData[idx].pIdx].nIdx = rIdx;
108             if (pData[rIdx].raw <= pData[pFilter->mIdx].raw) { pFilter->mIdx =
                ↪  pData[pFilter->mIdx].nIdx; }
109             pData[pData[idx].pIdx].nIdx = idx;
110             // Unlink the replaced element from its adjacent elements
111             pData[pData[rIdx].pIdx].nIdx = pData[rIdx].nIdx;
112             pData[pData[rIdx].nIdx].pIdx = pData[rIdx].pIdx;
113             // Link the new element to its new adjacent elements
114             pData[rIdx].pIdx = pData[idx].pIdx;
115             pData[rIdx].nIdx = pData[pData[idx].pIdx].nIdx;
116             pData[pData[rIdx].pIdx].nIdx = rIdx;
117             pData[pData[rIdx].nIdx].pIdx = rIdx;
118         }
119         // The number of elements matches the full size
120         size = pFilter->size;
121     }
122
123     // Fill the element
124     pData[rIdx].raw = value;
125
126     // Advance the replacement index
127     if (rIdx == pFilter->size) { pFilter->rIdx = 1; pFilter->state = 1; }
128     else { pFilter->rIdx++; }
129
130     // Compute the median
131     if (size & 1) // Odd number of elements (single value median)
132     {
133         pData[0].raw = pData[pFilter->mIdx].raw;
134     }
135     else // Even number of elements (two value average median)
136     {
```

```
137         pData[0].raw = ((pData[pFilter->mIdx].raw) +
            ↪  (pData[pData[pFilter->mIdx].nIdx].raw)) / 2;
138     }
139
140     // Return successfully
141     return pData[0].raw;
142 }
```

En la figura 7 se puede observar los resultados de una prueba realizada para comparar el desempeño del filtro de mediana con el filtro de media móvil exponencial (EWMA). En el eje Y se tiene el ángulo en grados traducido desde la tensión del potenciómetro (método explicado posteriormente), en el eje X se tiene el número de muestra tomada, donde cada muestra se encuentra separada 10ms. En naranja se tiene la señal target que el motor debe seguir; en amarillo se tiene la señal cruda obtenida del potenciómetro; en celeste la salida del filtro de media móvil exponencial con $\alpha = 0{,}1$; en rosa la salida del filtro de mediana con una ventana de 20 muestras. Ambos filtros corriendo a la misma frecuencia de sampleo, 200Hz.

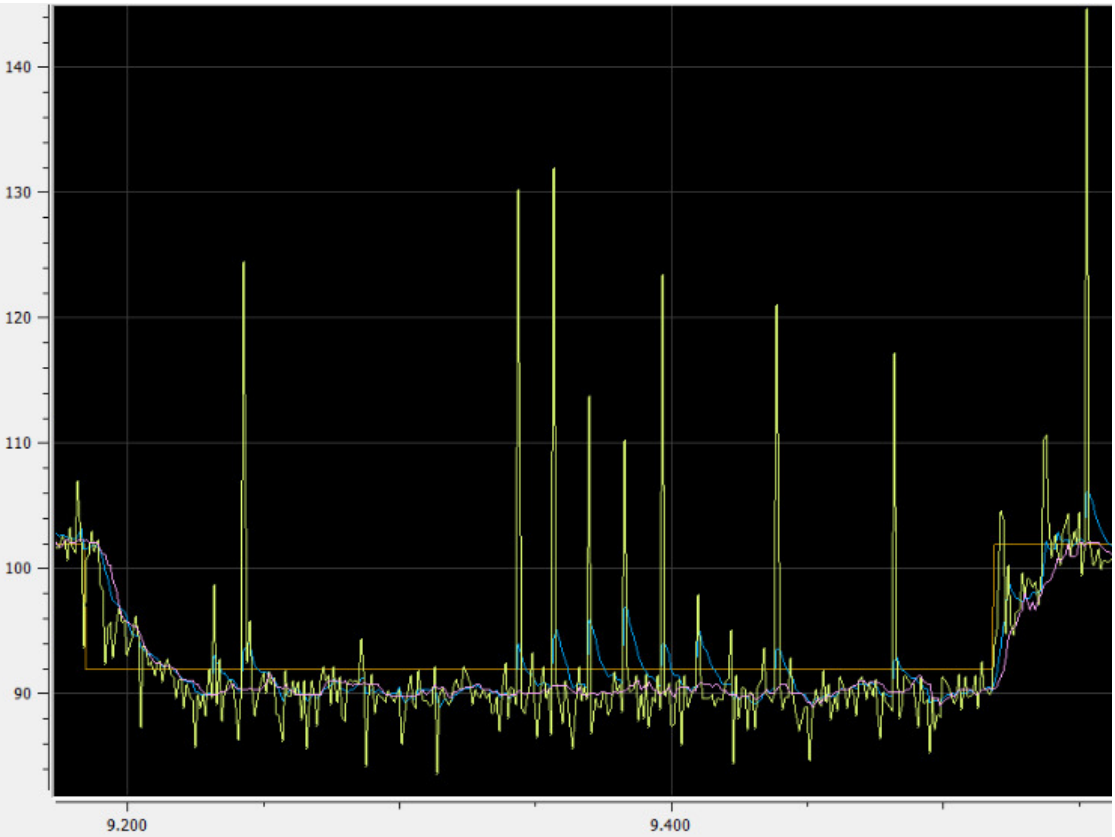


Figura 7: Comparación de filtro de mediana y media móvil exponencial

Se puede observar que el EWMA fue ajustado para seguir correctamente los flancos de

la señal pero tiene problemas para eliminar los picos de ruido elevado. Mientras tanto, el filtro de mediana es capaz de eliminarlos completamente, sin perder la forma de los flancos de la señal original.

Es importante tener en cuenta la capacidad del filtro de mediana para eliminar los picos depende del tamaño de la ventana. Para que el ruido sea ignorado correctamente debe representar menos de la mitad de las muestras de la ventana. Por esta razón, el tamaño temporal de la ventana debe ser mayor al doble del ancho de los picos de ruido. Por otro lado, la demora del filtro es igual a la mitad del ancho de la ventana. Esto hace que la ventana deba ser lo suficientemente grande como para ignorar el ruido completamente, pero lo más chica posible para minimizar la demora que puede traer problemas al sistema de control. Se optó por aumentar la frecuencia de sampleo del filtro a 2000Hz para tener mayor resolución en la medición y granularidad del ancho de la ventana, y ensanchar la ventana a 40 muestras, obteniendo una demora de 10ms, en lugar de los 50ms de demora que tiene el filtro de la figura.

4.2.2. Calibración

Para traducir la tensión del potenciómetro a ángulo del servomotor se podría medir la tensión con 0°, y la tensión con 270° y asumir un comportamiento lineal del potenciómetro para todos los valores intermedios. Sin embargo, con pruebas anteriores que se realizaron con otros servos, se observó que la curva que relaciona ángulo del rotor con tensión no resultaba lineal, y además cambia dependiendo si gira en sentido horario o antihorario, es decir, tiene histéresis y no es la misma para cada servomotor. Por este motivo, se llegó a la conclusión de que es necesario realizar una calibración de cada servomotor tomando varias mediciones de tensión para varios valores de ancho de pulso de la señal PWM, tanto en subida como en bajada. Teniendo en cuenta que estas mediciones se realizan en vacío (sin conectar nada en el eje), las señales de PWM son traducidas a posiciones angulares mediante una relación lineal. Posteriormente se corroboró que se cumpla esta linealidad, utilizando un encoder magnético para obtener la verdadera posición angular para cada valor de PWM. Para realizar la calibración, se realizó una máquina de estados programada en C que funciona en una NUCLEOF401RE, que varía linealmente el duty del PWM del servo, y toma 5 muestras del filtro de mediana que se promedian y almacenan en un buffer.

```
1  void State_Machine_Calibration(void)
2  {
3      static uint8_t state = UPDATE_PWM;
4      static uint16_t pwm_value = 0;
5      static uint16_t ton = 0;
6      static int8_t calibration_direction = 1;
7      static uint8_t cant_med = 0;
8      static float adc_acum = 0.0;
9      static float encoder_acum = 0.0;
10
11     switch (state)
```

```
{
    case UPDATE_PWM:

        ton = ton_pwm_calibration[pwm_value];

        __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2,ton);
        HAL_Delay(1000);
        time = ONE_SECOND;

        state = MEASURE;
        break;
    case MEASURE:
        if (cant_med == N_SAMPLES)
        {
            cant_med = 0;
            if (calibration_direction == 1)
            {
                adc_values_up[pwm_value] = adc_acum;
            }

            else if ( calibration_direction == -1)
            {
                adc_values_down[pwm_value] = adc_acum;
            }

            adc_acum = 0;
            encoder_acum = 0;

            if(calibration_direction == -1 && pwm_value == 0)
            {
                state = CALIBRATION_FINISHED;
                break;
            }

            pwm_value += calibration_direction;

            if(pwm_value == MED_CALIB)
            {
                pwm_value = MED_CALIB -1;
                calibration_direction = -1;
            }

            state = UPDATE_PWM;
        }
        else if (time == 0)
        {
            adc_acum += median_filteredValue;
            time = ONE_SECOND;
            cant_med++;
        }
        break;
    case CALIBRATION_FINISHED:
        HAL_Delay(1);
```

```
            break;
    }
}
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *timer)
{
    if (timer == &htim2)
    {
        if (time > 0)
            time--;

        if (HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY) == HAL_OK)
        {
            potentiometer_value = HAL_ADC_GetValue(&hadc1);
            HAL_ADC_Start(&hadc1);
        }
        median_filteredValue = MedianFilter(&median_filter, potentiometer_value);
    }
}
```

Con estas mediciones se armaron 24 tablas, una tabla de valores con el rotor girando en sentido horario y otra en sentido antihorario para cada servomotor, con 20 mediciones cada una. Luego de realizar la calibración, se observó que estos servomotores no poseen prácticamente ninguna histéresis y que la respuesta es suficiente lineal, siendo que la regresión lineal que se realizó dio $R^2 = 1$ (Fig. 8).

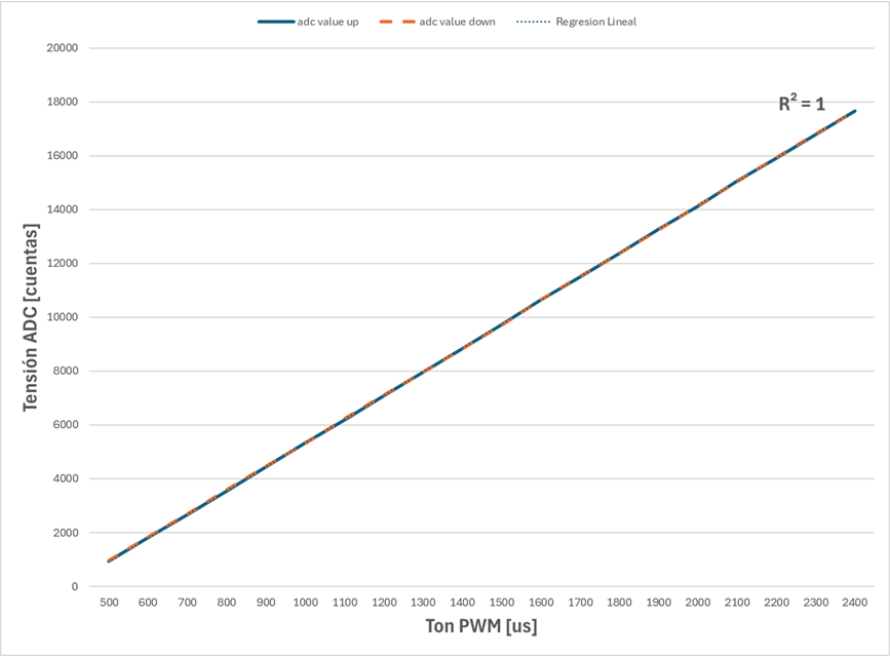


Figura 8: Variación de la tensión del potenciometro cuando el duty del PWM varía linealmente de 500us a 2400us.

Se realiza una interpolación lineal para obtener valores entre medio de cada medición, obteniendo así un ángulo para cada valor de tensión del ADC.

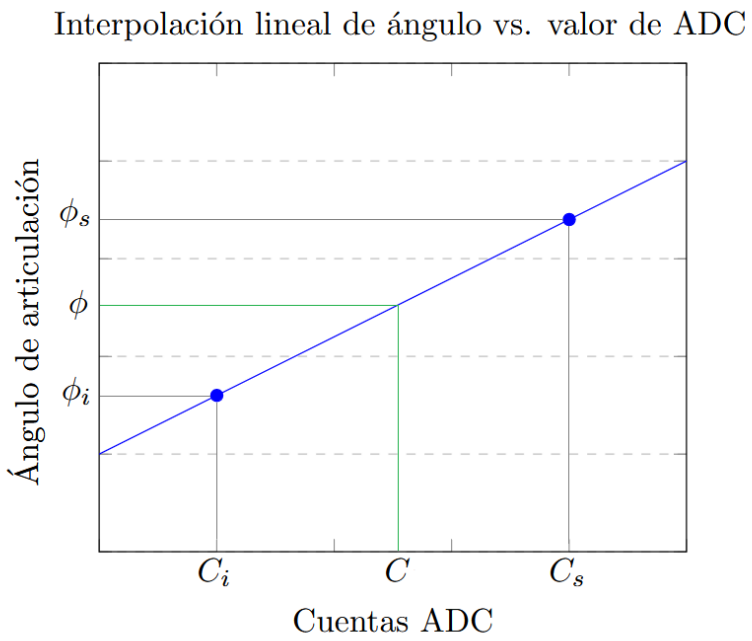


Figura 9: Interpolación lineal para obtener valores intermedios.

Usando como referencia la figura 9, se utiliza la siguiente expresión para averiguar el valor ϕ correspondiente a la cuenta C obtenida del ADC, sabiendo que dicha cuenta se encuentra entre dos mediciones de la tabla, una inferior (C_i, ϕ_i) y una superior (C_s, ϕ_s):

$$\phi(C) = \frac{\phi_i(C_s - C) + \phi_s(C - Ci)}{Cs - Ci}$$

El código implementado es el siguiente:

```
float adc2angle(uint16_t adc_value, uint8_t up_down, const float32_t *table_ptr)
{
    uint16_t index = 0;

    const float32_t *ADC_VALUES = table_ptr;

    // Busco el código en la tabla obtenida en la calibracion
    while (adc_value > ADC_VALUES[up_down * MED_CALIB + index] / 5)
    {
        index++;
         //adc_value mayor que el máximo valor de la tabla
        if (index > MED_CALIB - 1)
          return 266;
```

```
    }
    // Si el código está en la tabla, devuelvo el ángulo correspondiente
    if (adc_value == ADC_VALUES[up_down * MED_CALIB + index] / 5)
        return ANGLES[index];

    // adc_value menor que el mínimo valor de la tabla
    else if (index == 0)
        return 0;

    // Si está entre 2 códigos de la tabla realizo una interpolacion lineal
    else
        return (ANGLES[index - 1] * (ADC_VALUES[up_down * MED_CALIB + index] / 5 -
        ↪  adc_value) + ANGLES[index] * (adc_value - ADC_VALUES[up_down * MED_CALIB +
        ↪  index - 1] / 5)) / (ADC_VALUES[up_down * MED_CALIB + index] / 5 -
        ↪  ADC_VALUES[up_down * MED_CALIB + index - 1] / 5);
}
```

4.2.3. Control Servomotores

Los servomotores, al estar exigidos a diferentes momentos de inercia, tardan un cierto tiempo en llegar o no al ángulo target. Se podría implementar un sistema en el que una vez que se establecen todas las señales PWM de los servos, se espera un tiempo arbitrario, y luego se obtienen los ángulos en ese momento. Esto no es óptimo ya que los servos (dependiendo de las condiciones) pueden tardar distinto tiempo en llegar al target, y puede haber cierto error en la posición final que no sería contemplado. Por lo tanto, un enfoque más optimizado sería monitorear cada cierto tiempo el valor de los ángulos y decidir con cierto criterio, si cada articulación llegó a la posición final o no. Puede ocurrir que una articulación nunca llegue al valor deseado, por lo que es conveniente darle un tiempo máximo al robot para que todas sus articulaciones lleguen al target, y evitar que este se quede indefinidamente bloqueado.

Por otro lado también se detectó una diferencia entre el movimiento de los actuadores con y sin carga. En otras palabras, existe un error en el ángulo final al que llegan los servomotores cuando el robot está apoyado en el suelo y debe levantar su propio peso comparado al ángulo al que llegan cuando el robot está suspendido en el aire. Para solucionarlo se implementó un controlador proporcional e integral (PI), que determina el valor del PWM para ajustar la posición angular de los servomotores en base al error entre el ángulo medido y el ángulo target.

En la figura 10 se muestra un diagrama en bloques que contempla todo el sistema de control del robot, tanto la política que genera los ángulos target con la red neuronal entrenada, como el control PI que controla los servomotores a partir de realimentar los ángulos de cada articulación obtenidos con el filtro de mediana y las tablas de calibración, y una máquina de estados que supervisa constantemente cada paso discreto del Proceso de Decisión de Markov, que modela la interacción del agente con el entorno.

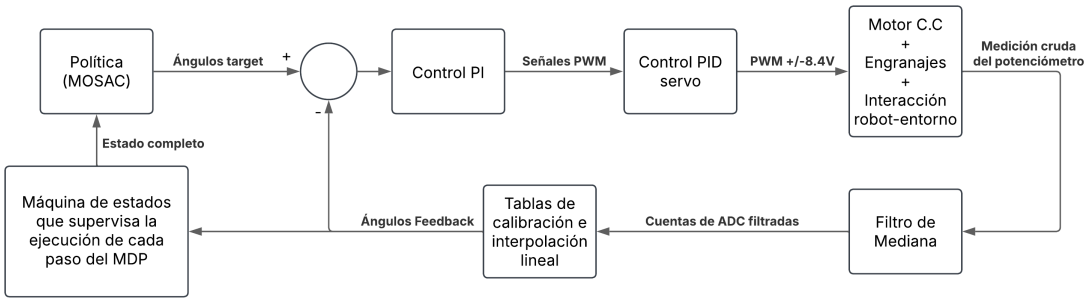


Figura 10: Diagrama de bloques del sistema de control.

En la figura 11 se presenta un diagrama de flujo que detalla el funcionamiento de la máquina de estados que revisa periódicamente los ángulos de las articulaciones para determinar si estas llegaron al target o no.

Inicialmente se calcula la diferencia angular de cada articulación con su respectivo target (Δ**target**), para saber si el servo será capaz de mover la articulación o no, debido al dead bandwidth de 0.27° mencionado anteriormente. Este valor luego fue modificado a un valor mayor para incentivar a la inteligencia artificial a realizar movimientos más amplios. Si la diferencia resulta mayor a este valor, quiere decir que el servomotor debe moverse, por lo que se actualiza una variable de 12 bits llamada ***joints_finished***, que lleva un registro de las articulaciones que han alcanzado su target: si el bit i está en '1', la articulación i alcanzó el target. Esta verificación se realiza para todos los motores. En este punto, si ninguna articulación debe moverse, es decir, **joints_finished** vale **ALL_FINISHED** (o 0xFFF), se retorna 1, y el paso concluye. Caso contrario, se reinicia los buffers que utiliza el controlador PID y su flag que indica que hay disponible una nueva muestra del mismo (**pid_sample**), comienza una cuenta regresiva el timer que controla el timeout y se reinicia el flag ***step_complete*** que indica si el paso terminó. Se espera por una nueva muestra del PID (**pid_sample == 1**). Cuando esto ocurre, se vuelve a calcular el Δtarget y esta vez se compara si esta diferencia es menor que el valor **MAX_DELTA_TARGET**. Este representa el error máximo que se está dispuesto a tolerar entre la posición final de la articulación y el target. En caso que haya llegado a su posición final dentro del margen $\pm$MAX_DELTA_TARGET, se actualiza **joints_finished** en el bit correspondiente y se corrobora si todas las articulaciones terminaron su paso (return 1 que significa **step_complete = 1**). Luego de realizar este procedimiento para todos los actuadores, se vuelve a esperar por una nueva muestra del PID y el proceso se repite. En todo momento, si la cuenta regresiva del timeout concluyó antes de que **joints_finished** valga **ALL_FINISHED**, también se finaliza el paso (return -1 que significa **step_complete = -1**).

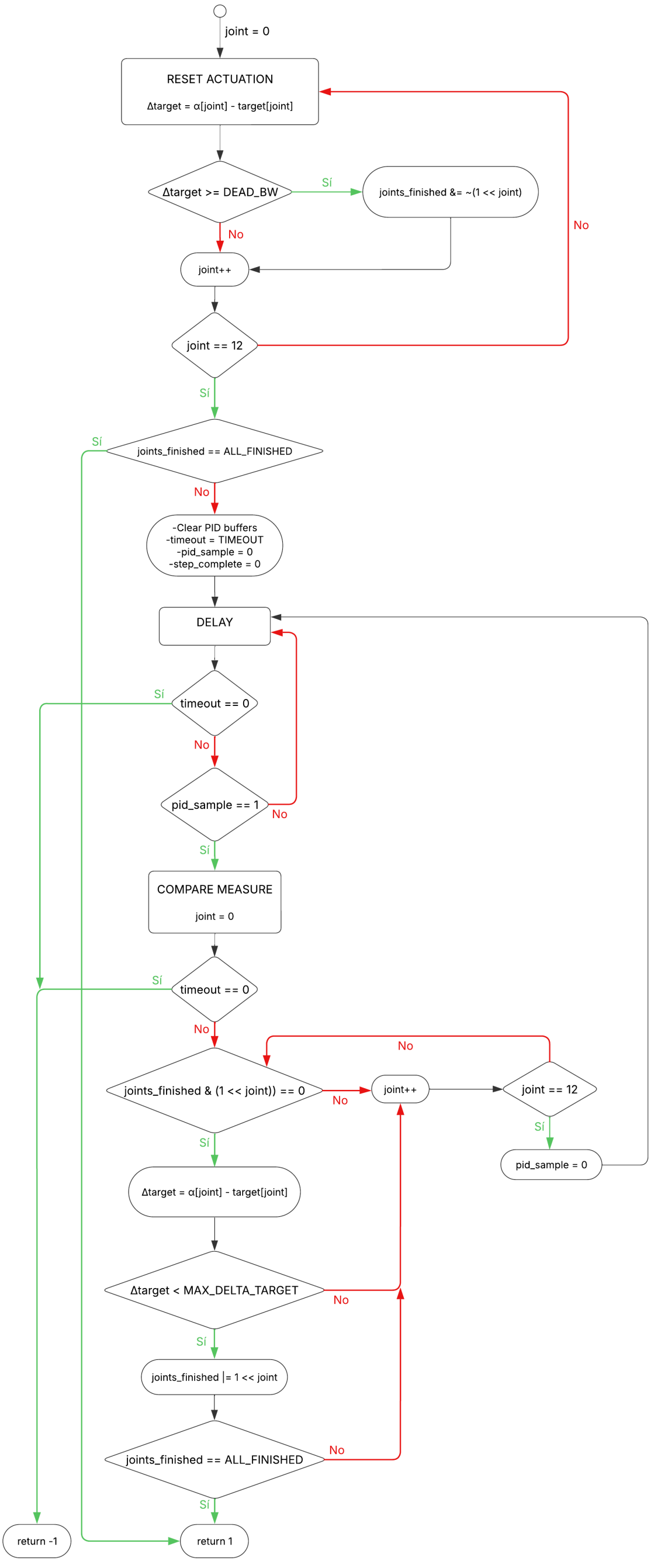


Figura 11: Diagrama de flujo de State Machine Actuation.

El código que implementa el diagrama de flujo anterior es el siguiente:

```
int8_t State_Machine_Actuation(void)
{
  switch (state_actuation)
  {
  case RESET_ACTUATION:

    //Clip targets, and evaluate which joints to move
    for (uint8_t joint = 0; joint < JOINTS; joint++)
    {

      delta_target = fabs(f_joint_angle[joint] - target_joint[joint]);

      if (delta_target >= DEAD_BANDWIDTH_SERVO)
      {
        // Evaluate if servo has to go up or doum to get a better measurement
        up_down_vector[joint] = (f_joint_angle[joint] < target_joint[joint]);

        joints_finished &= ~(1 << joint); // Set respective bit in 0
      }
    }

    if (joints_finished == ALL_FINISHED) //No joint should move, skip actuation
    {
      HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, 0);
      HAL_GPIO_WritePin(LD1_GPIO_Port, LD1_Pin, 1);
      return 1;
    }
    else
    {
      // Clear PID buffers
      memset((void*)error_acum, 0, sizeof(error_acum));
      memset((void*)previous_error, 0, sizeof(filt_angle_ADC));

      state_actuation = DELAY_STATE;
      timeout = TIMEOUT;
      pid_sample = 0;
      step_complete = 0;
    }
    break;

  case DELAY_STATE:
    if (timeout == 0)
      state_actuation = TIMEOUT_STATE;
    else if (pid_sample)
      state_actuation = COMPARE_MEASURE;

    break;
  case COMPARE_MEASURE:
    if (timeout == 0)
      state_actuation = TIMEOUT_STATE;

```

```
52      else
53      {
54        for (uint8_t joint = 0; joint < JOINTS; joint++)
55        {
56          if ((joints_finished & (1 << joint)) == 0) // Ignore joints that finished
57          {
58              delta_target = fabs(f_joint_angle[joint] - target_joint[joint]);
59
60              if (delta_target < MAX_DELTA_TARGET) // If the stable joint reached target
61              {
62                joints_finished |= 1 << joint; // Set respective bit in 1
63
64                if (joints_finished == ALL_FINISHED)
65                {
66                  state_actuation = RESET_ACTUATION;
67                  HAL_GPIO_WritePin(LD1_GPIO_Port, LD1_Pin, 1);
68                  return 1;
69                }
70              }
71
72          }
73        }
74        pid_sample = 0;
75        state_actuation = DELAY_STATE;
76      }
77
78      break;
79    case TIMEOUT_STATE:
80      state_actuation = RESET_ACTUATION;
81
82      HAL_GPIO_WritePin(LD1_GPIO_Port, LD1_Pin, 0);
83      HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, 1);
84
85      // Reset all joints
86      // joints_finished = ALL_FINISHED;
87      for (uint8_t joint = 0; joint < JOINTS; joint++)
88      {
89        //__HAL_TIM_SET_COMPARE(htim[joint / 4], channel[joint % 4], angle2ton_us(90));
90      }
91      return -1;
92    }
93    return 0;
94  }
```

El controlador PI se ejecuta en un timer que corre a una frecuencia de 100Hz. Se ha detectado haciendo pruebas que los actuadores a veces se mueven o vibran cuando ya el paso se completó y se está esperando por un nuevo target, lo cual empeora la caminata del robot. Por eso mismo, el PI se desactiva con el flag **step_complete** una vez que se completó el paso.

Por otra parte, para evitar que los servomotores lleven a las articulaciones a posicio-

nes que puedan dañarlas debido a fluctuaciones de las señal de control del PI o torques externos repentinos, se corrobora que el PWM de salida del control traducido a ángulo, esté dentro del rango de operación seguro del robot.

Se detalla a continuación el código del controlador PI:

```
for (uint8_t joint = 0; joint < 12; joint++)
{
    f_joint_angle[joint] = adc2angle(median_filteredValue[joint],
    ↪  up_down_vector[joint], calibration_table[joint]);

    if (step_complete == 0)
    {
        error = target_joint[joint] - f_joint_angle[joint];
        error_acum[joint] += error * TIM9_TICK;
        error_dif = (error - previous_error[joint]) / TIM9_TICK;

        previous_error[joint] = error;

        control_signal = kp[joint] * error + ki[joint] * error_acum[joint] + kd[joint]
        ↪  * error_dif;
        up_down_vector[joint] = control_signal > 0;

        pwm_pid_out[joint] = pwm_pid_out[joint] + __round_int(control_signal);

        //If pid_out is below min range, move to min range
        if(ton_us2angle(pwm_pid_out[joint]) < (mid_point_joints[joint] +
        ↪  joint_range[joint][0]))
            move_servos(joint, angle2ton_us(mid_point_joints[joint] +
            ↪  joint_range[joint][0]));

        //If pid_out is above max range, move to max range
        else if(ton_us2angle(pwm_pid_out[joint]) > (mid_point_joints[joint] +
        ↪  joint_range[joint][1]))
            move_servos(joint, angle2ton_us(mid_point_joints[joint] +
            ↪  joint_range[joint][1]));

        //Else move it to pid_out
        else
            move_servos(joint, pwm_pid_out[joint]);
    }
}
pid_sample = 1;

```

En la tabla 2 se muestran los valores de los parámetros utilizados tanto para la máquina de estados como para el control PI.

Parámetro	Valor	Unidades
TIM5 TICK	0,5	ms
TIM9 TICK	0,01	s
DEAD BANDWIDTH SERVO	3	°
MAX DELTA TARGET	2	°
TIMEOUT	1000	ms
WINDOW SIZE	80	$muestras$
KP	0,25	–
KI	0,1	–

Tabla 2: Parámetros control servomotores.

4.2.4. Filtro de Kalman

Como se mencionó en la sección 4.1.5, el estado del agente está compuesto por la posición angular de todas las articulaciones, la posición (x, y, z) y **los ángulos de inclinación** con respecto a los planos coordenados del sistema de referencia inercial fijo del entorno, y la velocidad angular de dichos ángulos.

Estos se denominan Yaw, Pitch y Roll, o Ángulos de Euler, y miden la inclinación del sistema de referencia de un cuerpo móvil con respecto a un sistema de referencia inercial.

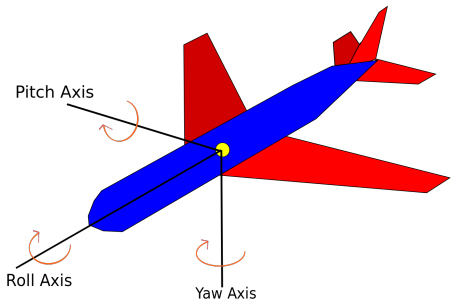


Figura 12: Movimiento en Yaw, Pitch, Roll de un cuerpo en movimiento.

Existen diferentes métodos para medir estos ángulos. Teniendo como referencia la aceleración de la gravedad apuntando hacia abajo y midiendo las componentes vectoriales de esta misma aceleración desde el punto de vista del agente, es posible determinar la inclinación que tiene el cuerpo con respecto al plano normal de la superficie terrestre. Esta inclinación puede ser tanto en Pitch como en Roll. Por lo tanto, colocando un **sensor acelerómetro** en el cuerpo móvil es posible determinar estos ángulos. Sin embargo, lo antes mencionado es solamente cierto si el cuerpo se encuentra en un estado de cuasi-reposo y no sufre de aceleraciones lineales considerables que afecten la medición del vector de gravedad. Por lo tanto se dice que **esta medición sirve únicamente**

para variaciones en la inclinación lentas o de baja frecuencia. Notar que este método **no sirve para estimar el ángulo Yaw**.

Para la medición de variaciones rápidas en la inclinación, se puede medir la velocidad angular en los 3 ejes del cuerpo móvil con un **sensor giróscopo**, y luego realizar una integración numérica durante un cierto tiempo para medir cuál fue el desplazamiento angular. El tiempo de integración será el período que haya entre muestras adquiridas. El problema que tiene este tipo de medición de ángulos es la acumulación de error con el tiempo. Estos tipos de sensores denominados MEMS (Micro Electro-Mechanical System), son muy ruidosos, por lo que al integrar su medición para obtener los ángulos, también se va acumulando un error que provoca que la medición se desplace de su valor verdadero; un efecto conocido como "measurement drift". Por lo tanto, **esta medición sirve únicamente para variaciones en la inclinación rápidas o de alta frecuencia**, que no requieren períodos largos de integración. Notar que este método **sirve para estimar el ángulo Yaw de solo de forma relativa**, ya que no se tiene ningún tipo de referencia sobre el Norte de la Tierra (es necesario un magnetómetro) para corregir la medición.

Por lo tanto, para obtener una buena estimación de Pitch y Roll, se debe poder combinar la medición del acelerómetro de baja frecuencia con la medición del giróscopo de alta frecuencia. Existen diferentes técnicas de fusión de sensores, denominadas "Sensor Fusion". Una de las más utilizadas es el Filtro Complementario el cual adquiere las mediciones de los dos sensores y realiza una suma pesada (o combinación lineal). Esta suma pesada le dará más importancia a la medición del acelerómetro o a la del giróscopo. Esta técnica es útil cuando se sabe a priori si el sistema va a tender a comportarse más lento (caso en el que la medición del acelerómetro es más importante que la del giróscopo) o más rápido (medición del giróscopo más importante que la del acelerómetro), ya que el coeficiente que pesa las mediciones es constante. Pero cuando el sistema puede tener diferentes momentos donde las variaciones de ángulo pueden ser rápidas o lentas, esta técnica no es viable.

Una técnica más avanzada que la anterior se denomina ***"Filtro de Kalman"*** [8], y la particularidad que tiene es la **variación automática del coeficiente de la suma pesada** mencionada anteriormente. Dependiendo como esté evolucionando el sistema, el filtro de Kalman ajustará automáticamente este coeficiente para darle más importancia a una medición que a otra. Este coeficiente es una matriz denominada ***"Ganancia de Kalman"***.

El filtro de Kalman tiene tres pasos:

1. Inicialización de los estados; Yaw, Pitch y Roll.
2. Predicción de los estados a partir del giróscopo e Integración de Euler.
3. Corrección de los estados a partir del acelerómetro.

El algoritmo, luego de inicializar los estados, itera entre el paso de predicción y corrección.

La predicción está dada por el modelo físico del sistema, es decir, si se conoce la velocidad angular en los 3 ejes y las ecuaciones que gobiernan las rotaciones del cuerpo, es posible predecir cuál será el siguiente estado.

La corrección está dada por la comparación de la predicción del paso anterior, con la medición del acelerómetro. Cabe aclarar que, para esta aplicación, los estados del filtro de Kalman no son los ángulos Yaw, Pitch y Roll, sino su representación en forma de cuaterniones. Esto es así para evitar errores en el proceso de cálculo como el "Gimbal Lock".

Esquemáticamente el Filtro de Kalman se puede ver como en la figura 13:

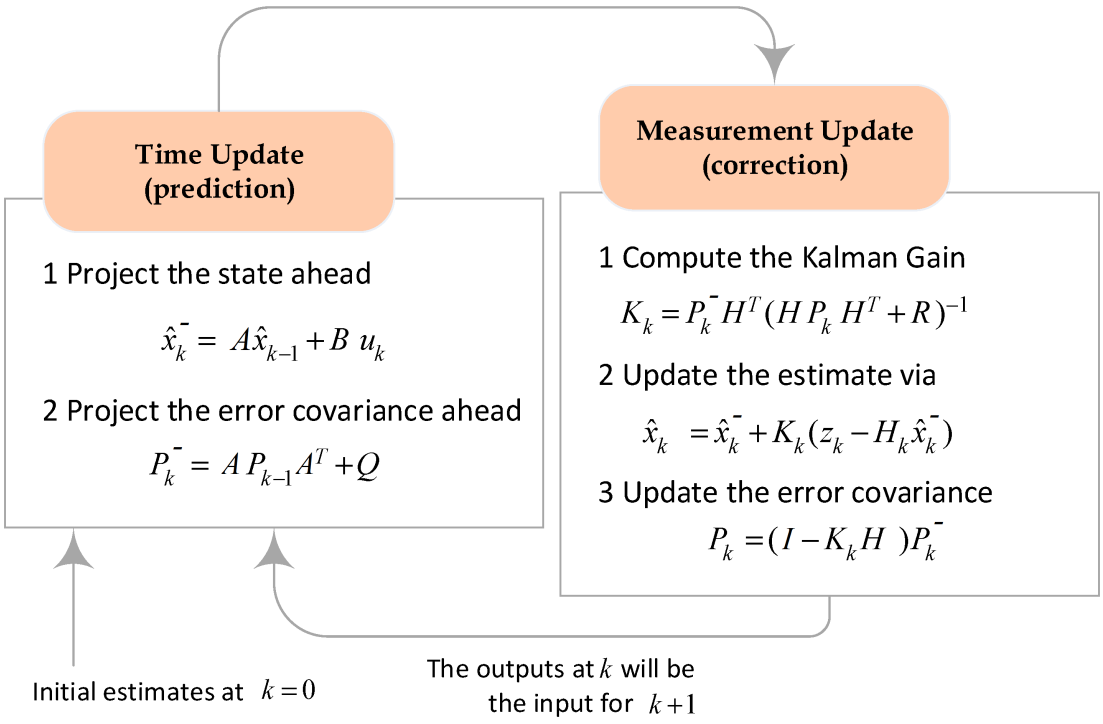


Figura 13: Diagrama de flujo Filtro de Kalman.

donde:

1. X_k^- es la estimación a priori del vector de estados.
2. X_k es el vector de estados corregido.
3. A es la matriz de transición de estados. Es la matriz que relaciona el estado actual con el estado futuro predicho.

4. P_k^- es la covarianza del error asociada a la estimación a priori.
5. P_k es la covarianza del error corregida.
6. z_k es el vector de mediciones al momento k.
7. R la matriz de covarianza del ruido de las mediciones (depende del acelerómetro).
8. Q es la matriz de covarianza del ruido asociado al modelo físico (depende del giróscopo).
9. H es la matriz que indica la relación entre mediciones y el vector de estado al momento k, en el supuesto ideal de que no hubiera ruido en las mediciones.
10. K la ganancia de Kalman.

La matriz A para este modelo se define como:

$$A = I + \frac{1}{2} \cdot dt \cdot \begin{bmatrix} 0 & -wx & -wy & -wz \\ wx & 0 & wz & -wy \\ wy & -wz & 0 & wx \\ wz & wy & -wx & 0 \end{bmatrix}$$

donde ω_x, ω_y y ω_z son las velocidades angulares medidas en el agente por el giróscopo, e I la matriz identidad de 4x4. El vector de estados al ser pre multiplicado por esta matriz devuelve el cuaternión representando la rotación estimada.

El vector z de mediciones para Roll & Pitch se define como la transformación a cuaternión de:

$$\begin{bmatrix} \theta \\ \phi \end{bmatrix} = \begin{bmatrix} arcsin(\frac{a_x}{g}) \\ arctanh(\frac{a_y}{a_z}) \end{bmatrix}$$

donde θ y ϕ son los estimadores de Roll y Pitch respectivamente, a_x, a_y y a_z las aceleraciones medidas en el robot por el acelerómetro. Estos ángulos de Euler luego son transformados en cuaterniones para corregir la estimación.

La matriz H para este modelo se define como la matriz identidad de 4x4. Dado que no hay diferencia entre el vector de mediciones y el vector de estado.

El proceso de predicción y estimación no necesariamente debe hacerse en una sola pasada del proceso iterativo, sino que pueden tener distinta frecuencia de ejecución. Dependerá de la frecuencia de muestreo de cada sensor.

Para estimar los ángulos de Euler del robot con respecto al sistema de referencia, se implementó una unidad de medición inercial (IMU - Inertial Measurement Unit) a partir de un sistema embebido **Seeed Studio XIAO nRF52840 Sense** [9] (Fig. 14) que

internamente dispone de un acelerómetro y giróscopo LSM6DS3TR-C. Posee un microcontrolador ARM Cortex-M4 de 32 bits con FPU, 64 MHz de CLK, un canal I2C, dos canales UART, 2 canales SPI, un canal I2S, y permite programarse con Arduino.

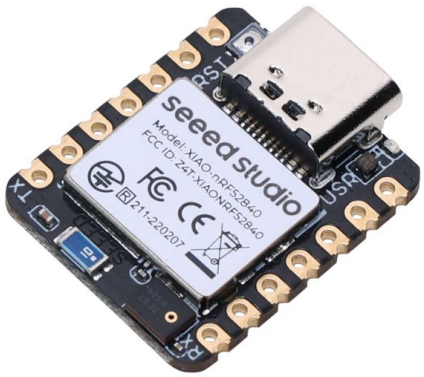


Figura 14: Sistema embebido utilizado para programa la IMU del robot.

Para obtener pitch y roll a partir de las mediciones del acelerómetro y giróscopo, se realizó un código programado en Arduino que implementa un filtro de Kalman. Las mediciones del acelerómetro y giróscopo son previamente filtradas con un FIR de media móvil, 20 taps para la aceleración y 3 para la velocidad angular. La aceleración debe tener un filtrado mayor ya que la señal es mucho más ruidosa que la de velocidad angular. Se hizo uso de las bibliotecas *FIR Filter* para el filtrado previo y *BasicLinearAlgebra* para todo el manejo de matrices del filtro de Kalman:

```
#include <BasicLinearAlgebra.h>
#include <ElementStorage.h>
#include <FIR.h>
#include <Wire.h>

#define G 9.80665f

Matrix<4,4,double>I = { 1,0,0,0,
                        0,1,0,0,
                        0,0,1,0,
                        0,0,0,1};
Matrix<4,4,double>H = I;
Matrix<4,4,double>K;
Matrix<4,4,double>A;
Matrix<4,4,double>Q = I * pow(1.2* PI/180,2);
Matrix<4,4,double>R = I * pow(0.002 * G,2);
//Matrix<4,4,double>Q = I * pow(0.028 * PI/180,2);
//Matrix<4,4,double>R = I * pow(0.0007 * G,2);
Matrix<4,4,double>P = I * 0.1;
Matrix<4,1,double>X = {1,
```

```
                        0,
                        0,
                        0};
Matrix<4,1,double>Z;


void loop() {
    ax_noisy = -myIMU.readFloatAccelX()*G;
    ay_noisy = myIMU.readFloatAccelY()*G;
    az_noisy = -myIMU.readFloatAccelZ()*G;

    wx_noisy = -(myIMU.readFloatGyroX()-gx_cal)*PI/180;
    wy_noisy = (myIMU.readFloatGyroY()-gy_cal)*PI/180;
    wz_noisy = -(myIMU.readFloatGyroZ()-gz_cal)*PI/180;

    Time = micros()/1000000.0f;
    dt = Time - Time_prev;
    Time_prev = Time;

    // FIR Filter First Order
    ax = fir_ax.processReading(ax_noisy);
    ay = fir_ay.processReading(ay_noisy);
    az = fir_az.processReading(az_noisy);

    wx = fir_wx.processReading(wx_noisy);
    wy = fir_wy.processReading(wy_noisy);
    wz = fir_wz.processReading(wz_noisy);

    A = { 0,   -wx,  -wy,  -wz,
          wx,  0,    wz,  -wy,
          wy,  -wz,   0,    wx,
          wz,   wy,  -wx,   0  };

    A = I + A*dt*0.5;

    X = A*X;
    P = A*P*(~A) + Q;

    angles = EP2Euler321(X);
    yaw = angles(0,0);

    Invert(H*P*(~H) + R,aux);
    K = P*(~H)*aux;

    angles = EulerAccel(ax, ay, az);
    angles(0,0) = yaw;

    Z = Euler3212EP(angles);

    X = X + K*(Z - H*X);
    P = P - K*H*P;

    angles = EP2Euler321(X);
```

```
74
75      yaw = angles(0,0);
76      pitch = angles(1,0);
77      roll = angles(2,0);
78
79      delay(1);
80  }
81
82  Matrix<3,1,double> EulerAccel(double ax, double ay, double az)
83  {
84    float g = sqrt(pow(ax,2) + pow(ay,2) + pow(az,2));
85
86    double theta = asin(ax/g);
87    double phi   = atan2l(-ay,-az);
88
89    Matrix<3,1,double> angles = {0,theta,phi};
90
91    return angles;
92  }
93
94  Matrix<3,1,double> EP2Euler321(Matrix<4,1,double> q)
95  {
96
97    /*E = EP2Euler321(Q) translates the Euler parameter vector
98    Q into the corresponding (3-2-1) Euler angle set.
99    */
100
101   double q0 = q(0,0);
102   double q1 = q(1,0);
103   double q2 = q(2,0);
104   double q3 = q(3,0);
105
106   Matrix<3,1,double> e;
107
108
109   e(0,0) = atan2(2*(q1*q2+q0*q3),q0*q0+q1*q1-q2*q2-q3*q3);
110   e(1,0) = asin(-2*(q1*q3-q0*q2));
111   e(2,0)= atan2(2*(q2*q3+q0*q1),q0*q0-q1*q1-q2*q2+q3*q3);
112
113   return e;
114 }
115
116 Matrix<4,1,double>Euler3212EP(Matrix<3,1,double> e)
117 {
118   /*
119   Q = Euler3212EP(E) translates the 321 Euler angle
120   vector E into the Euler parameter vector Q.
121   */
122   double c1 = cos(e(0,0)/2);
123   double s1 = sin(e(0,0)/2);
124   double c2 = cos(e(1,0)/2);
125   double s2 = sin(e(1,0)/2);
126   double c3 = cos(e(2,0)/2);
```

```
  double s3 = sin(e(2,0)/2);

  Matrix<4,1,double> q;

  q(0,0) = c1*c2*c3+s1*s2*s3;
  q(1,0) = c1*c2*s3-s1*s2*c3;
  q(2,0) = c1*s2*c3+s1*c2*s3;
  q(3,0) = s1*c2*c3-c1*s2*s3;

  return q;
}
```

En las figuras 15, 16 y 17 se muestran algunas mediciones obtenidas con la IMU y el filtro de Kalman. Si bien el Yaw presenta cierta deriva dado que solo se estima a partir de la integración del giróscopo, tanto el Pitch como el Roll presentan una buena respuesta temporal, sin ser afectados por la deriva, y siendo completamente independientes entre sí: la inclinación en uno de los ejes no afecta la medición en el eje perpendicular (si bien se introduce el error humano de inclinar manualmente la IMU durante el ensayo).

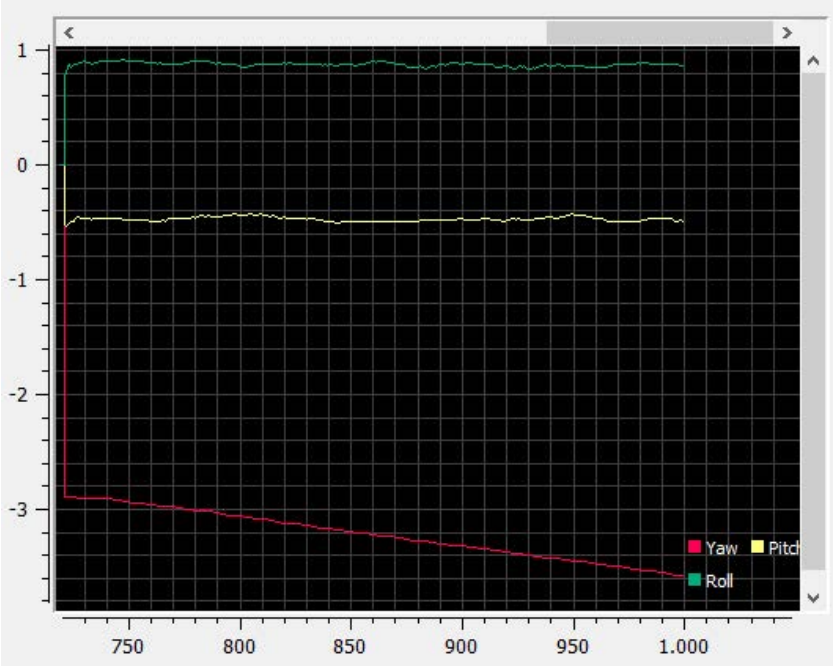


Figura 15: Yaw, Pitch & Roll sin ninguna rotación. Se observa la inclinación del entorno donde reposa la unidad de medición inercial.

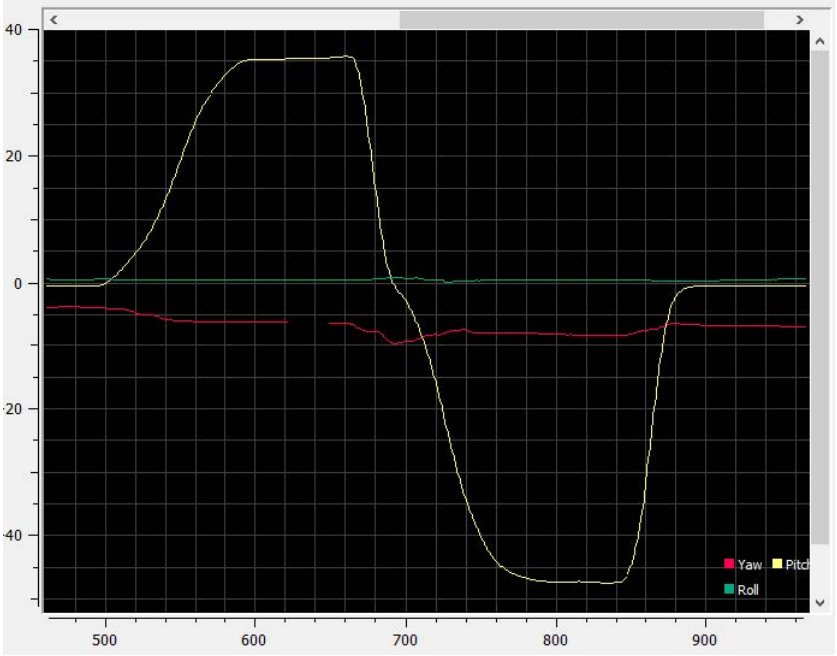


Figura 16: Yaw, Pitch & Roll rotación en Pitch.

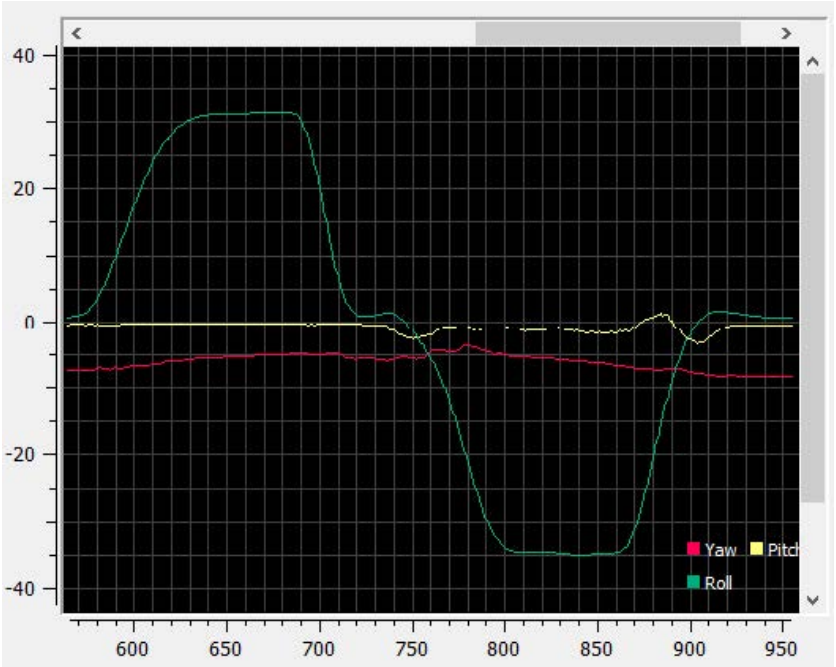


Figura 17: Yaw, Pitch & Roll rotación en Roll.

4.2.5. Ejecución de Política

La computadora principal del robot es una Raspberry Pi 4 con 8 GB de RAM [10], ejecutando una distribución de Linux basada en Debian. Su función principal es ejecutar un script en Python que implementa la red neuronal de la política. Este script gestiona la comunicación entre la Raspberry Pi, la placa NUCLEO y la XIAO, permitiendo la recepción de datos necesarios para conformar el estado e ingresarlos en la política. Además, se encarga de transmitir el vector de acciones a la NUCLEO, que a su vez controla los servomotores.

Dentro del vector de estados se encuentra la rotación objetivo del robot. Para generar este valor, se diseñó una aplicación móvil desarrollada en Dart utilizando el framework Flutter (Fig. 18). Esta aplicación proporciona una interfaz de usuario que permite gestionar diversos atributos del robot, como modificar preferencias, iniciar o detener su movimiento y ajustar la rotación target, entre otras funciones. Los datos se envían a la computadora mediante Bluetooth, utilizando la tecnología "Bluetooth Low Energy" (BLE).

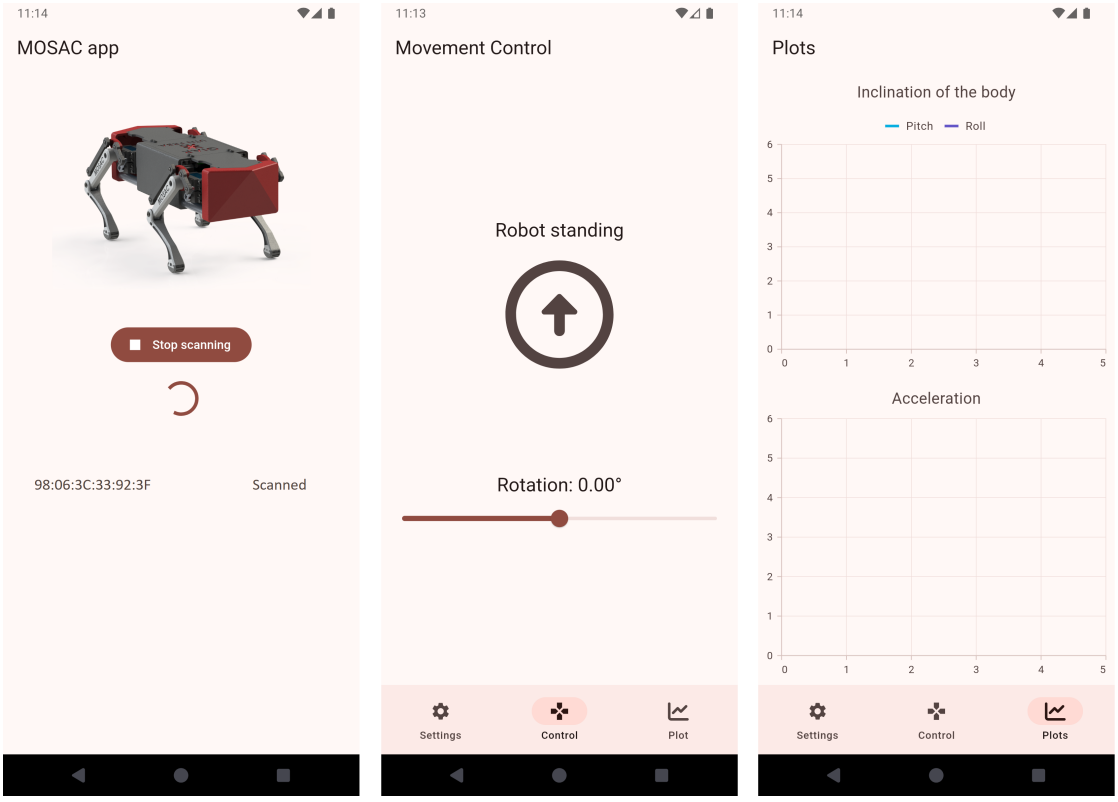


Figura 18: Capturas de la aplicación móvil de MOSAC.

La Raspberry Pi 4 establece comunicación con la XIAO a través de un canal I2C, mientras que la conexión con la NUCLEO se realiza mediante un canal SPI complementado con un GPIO adicional para sincronización (Fig. 19).

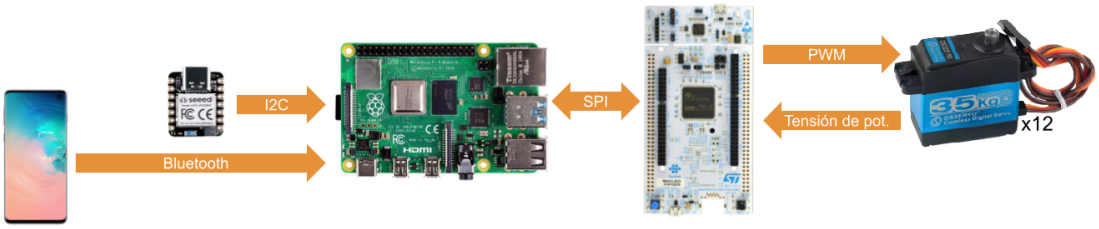


Figura 19: Conexión entre la Raspberry Pi, NUCLEO F412, XIAO nRF52840 Sense y App de Celular.

Se presenta a continuación el código que se ejecuta en la Raspberry Pi:

```
######Pinout information (https://pinout.xyz/)############
#SPI1 (supports 16 bit words):  MOSI (GPIO20, pin 38); MISO (GPIO19, pin 35); SCLK (GPIO21, pin 40)
#                               CE0 (GPIO18, pin 12), CE1 (GPIO17, pin 11), CE2 (GPIO16, pin 36)
#I2C1 (I2C0 is for the SD): SDA (GPIO2, pin 3); SCL (GPIO3, pin 5)
spi_slave_ready = 25 #GPIO25, pin 22

import pigpio #For GPIO, SPI and I2C
from Policy import P_Network
import numpy as np
import torch


pi = pigpio.pi()

##States##
state = "RX_SPI"

########SPI SETUP#######
rise_edge_detected = 0

def rising_edge_callback(gpio, level, tick):
    global rise_edge_detected
    if gpio == spi_slave_ready:
        rise_edge_detected = 1
        print("rise: ", tick)

pi.set_mode(spi_slave_ready, pigpio.INPUT)
pi.callback(spi_slave_ready, pigpio.RISING_EDGE, rising_edge_callback)

#spi_flags:
#21 20 19 18 17 16 15 14 13 12 11 10  9  8  7  6  5  4  3  2  1  0
```

```
# b  b  b  b  b  b  R  T  n  n  n  n  W  A u2 u1 u0 p2 p1 p0  m  m

#bbbbbb defines the word size in bits (0-32)
#R and T = 0 means msb first for both MISO and MOSI
#W = 0 means SPI is not 3-wire (MISO and MOSI in different lines)
#A is 0 for the main SPI, 1 for the auxiliary SPI
#Ux = 1 means CEx disabled
#px sets if CEx is active low or high
#mm defines the mode

spi_flags = 0b01000000000000111100000
hspi = pi.spi_open(0,500000,spi_flags)    #Auxiliary SPI, 16 bits per word, msb first,
↪  mode 0, without CE, 1MHz

#######I2C SETUP########
IMU_address = 0x1
hi2c = pi.i2c_open(1, IMU_address, 0)   #No configuration posible for I2C (not needed)


#----Observation vector----#
#   target step rotation of the agent's body
#   pitch and roll of the agent's body
#   pitch and roll angular velocities of the agent's body
#   the 12 joint angles
obs_dim=17

#----Action vector----#
#   the 12 joint angles (pwm active time for each servo)
act_dim=12

#----Preference vector----#
#   [vel_forward, acceleration, vel_lateral, orientation, flat_back, energy, paws]
pref_dim = 7

first_time = 1

#----Joint Denormalization----#
body_min, body_max = -10.0, 15.0
body_mean = (body_min + body_max)/2
body_range = (body_max - body_min)/2

leg_min, leg_max = -30.0, 30.0
leg_mean = (leg_min + leg_max)/2
leg_range = (leg_max - leg_min)/2

paw_min, paw_max = -15.0,  15.0
paw_mean = (paw_min + paw_max)/2
paw_range = (paw_max - paw_min)/2

cant_steps = 0

if __name__ == '__main__':
    P_net = P_Network(obs_dim, act_dim, pref_dim, hidden1_dim=64, hidden2_dim=32)
```

```
    P_net.load_checkpoint()

    pref = torch.tensor([[2, 1, 1, 2, 1, 0, 0]]).to(P_net.device)

    while True:

        match state:
            case "RX_SPI":    ###Receive servo angles from Nucleo-F412ZG with SPI1
                if rise_edge_detected:
                    rise_edge_detected = 0
                    print("rise=0")
                    first_time = 1

                    (bytes_read, joints_byte_list) = pi.spi_read(hspi, 12*4)
                    if bytes_read == 12*4: print("Se leyó todo")
                    else: print("no se leyeron la cantidad de bytes correcta - Nucleo")

                    measured_joints_nucleo = np.frombuffer(bytes(joints_byte_list),
                    ↪  dtype='<f4')   #< little endian, > big endian
                    print("measured_joints_nucleo = ", measured_joints_nucleo)

                    #Change the order from webots to the order the NUCLEO needs based
                    ↪  on servo conections to the timers:
                    measured_joints = np.zeros(12,dtype = np.float64)

                    # Body  Front   Right
                    measured_joints[0] = measured_joints_nucleo[0]
                    # Femur   Front   Right
                    measured_joints[1] = measured_joints_nucleo[4]
                    # Tibia   Front   Right
                    measured_joints[2] = measured_joints_nucleo[3]
                    # Body  Front   Left
                    measured_joints[3] = measured_joints_nucleo[5]
                    # Femur   Front   Left
                    measured_joints[4] = - measured_joints_nucleo[6]
                    # Tibia   Front   Left
                    measured_joints[5] = - measured_joints_nucleo[7]
                    # Body  Back    Right
                    measured_joints[6] = - measured_joints_nucleo[1]
                    # Femur   Back    Right
                    measured_joints[7] = measured_joints_nucleo[11]
                    # Tibia   Back    Right
                    measured_joints[8] = measured_joints_nucleo[10]
                    # Body  Back    Left
                    measured_joints[9] = - measured_joints_nucleo[2]
                    # Femur   Back    Left
                    measured_joints[10] = - measured_joints_nucleo[8]
                    # Tibia   Back    Left
                    measured_joints[11] = - measured_joints_nucleo[9]

                    #Normalization
                    for i in range(4):
                        measured_joints[i*3] =      (measured_joints[i*3]    -
                        ↪  body_mean)   /   body_range
```

```
                        measured_joints[1+i*3] =    (measured_joints[1+i*3]  -
                        ↪  leg_mean)    /   leg_range
                        measured_joints[2+i*3] =    (measured_joints[2+i*3]  -
                        ↪  paw_mean)    /   paw_range

                    print("measured_joints = ", measured_joints)

                    state = "RX_I2C"

            case "RX_I2C": ###Receive IMU data from XIAO_NRF52840 with I2C

                #Read 4*4 bytes (4 floats)
                (count, data) = pi.i2c_read_device(hi2c, 4*4)
                if count == 4*4: print("Se leyó todo")
                else: print("no se leyeron la cantidad de bytes correcta - IMU")

                IMU_data = np.frombuffer(bytes(data), dtype='<f4')
                print("IMU_data = ", IMU_data)

                state = "POLICY"

            case "POLICY":
                cant_steps += 1
                # Assemble the state vector from the measurements of the NUCLEO and
                ↪  IMU
                target_rot = 0
                pitch = IMU_data[0]
                roll = IMU_data[1]
                vel_pitch = IMU_data[2]
                vel_roll = IMU_data[3]

                state = np.array([target_rot, pitch/np.pi, roll/np.pi, vel_pitch/np.pi,
                ↪  vel_roll/np.pi])
                state = np.concatenate((state, measured_joints), axis=0)
                state = np.expand_dims(state, axis=0)
                print("state", state)
                print("Pitch:" , pitch*180/np.pi)
                print("Roll:" , roll*180/np.pi)

                state = torch.tensor(state).to(P_net.device)

                #Compute the action with the policy network
                action, _ = P_net(state, pref)
                action = torch.tanh(action) #Restrict the actions to (-1;1)
                action = action.detach().cpu().numpy().reshape(-1).astype(np.float64)

                #Denormalize action with the ranges and midpoints of the joints
                for i in range(4):
                    action[i*3] = action[i*3] * body_range + body_mean
                    action[1+i*3] = action[1+i*3] * leg_range + leg_mean
                    action[2+i*3] = action[2+i*3] * paw_range + paw_mean

                #Change the order from webots to the order the NUCLEO needs based on
                ↪  servo connections to the timers:
```

```
                #Mid point is added inside the NUCLEO
                action_nucleo = np.zeros(12, dtype=np.float32)
                # BFR
                action_nucleo[0] = action[0]
                # BBR
                action_nucleo[1] = -action[6]
                # BBL
                action_nucleo[2] = -action[9]
                # TFR
                action_nucleo[3] = action[2]
                # FFR
                action_nucleo[4] = action[1]
                # BFL
                action_nucleo[5] = action[3]
                # FFL
                action_nucleo[6] = -action[4]
                # TFL
                action_nucleo[7] = -action[5]
                # FBL
                action_nucleo[8] = -action[10]
                # TBL
                action_nucleo[9] = -action[11]
                # TBR
                action_nucleo[10] = action[8]
                # FBR
                action_nucleo[11] = action[7]

                #Order of nucleo servos:
                #0PWM_BFR,1PWM_BBR, 2PWM_BBL, 3PWM_TFR, 4PWM_FFR, 5PWM_BFL, 6PWM_FFL,
                ↪  7PWM_TFL, 8PWM_FBL, 9PWM_TBL, 10PWM_TBR, 11PWM_FBR
                print("cant_steps = ", cant_steps)
                print("Action = ",          action)
                print("Action Nucleo = ", action_nucleo)

                #Convert float numpy array in byte list
                action_byte_list = list(action_nucleo.tobytes())

                state = "TX_SPI"

            case "TX_SPI":
                #Send Action with SPI
                if rise_edge_detected:
                    rise_edge_detected = 0
                    first_time = 1

                    pi.spi_write(hspi, action_byte_list)

                    state = "RX_SPI"

```

4.3. PCB

Para la interconexión de todas las placas, se diseñó un circuito impreso con el programa Altium 2024. Consta de dos conectores IDC de 50 pines cada uno para hacer llegar al PCB todas las señales necesarias desde y hacia la NUCLEO (PWM, ADC, SPI, GPIO); un conector IDC de 40 pines para conectar la Raspberry Pi 4; 4 borneras de 3 entradas y 12 tiras de pines de 3 posiciones para conectar las realimentaciones de posición y los PWM de los servomotores respectivamente; dos tiras de pines hembra para conectar la IMU en el centro de la placa; un conector USB para alimentar a la Raspberry y una bornera de alimentación. Esta última luego fue utilizada únicamente para alimentar los servomotores directamente de la batería, y se añadió otra bornera para alimentar desde una fuente buck-boost el resto de las placas. En la figura 20 y 21 se muestra el esquemático y PCB de la placa de interconexión respectivamente:

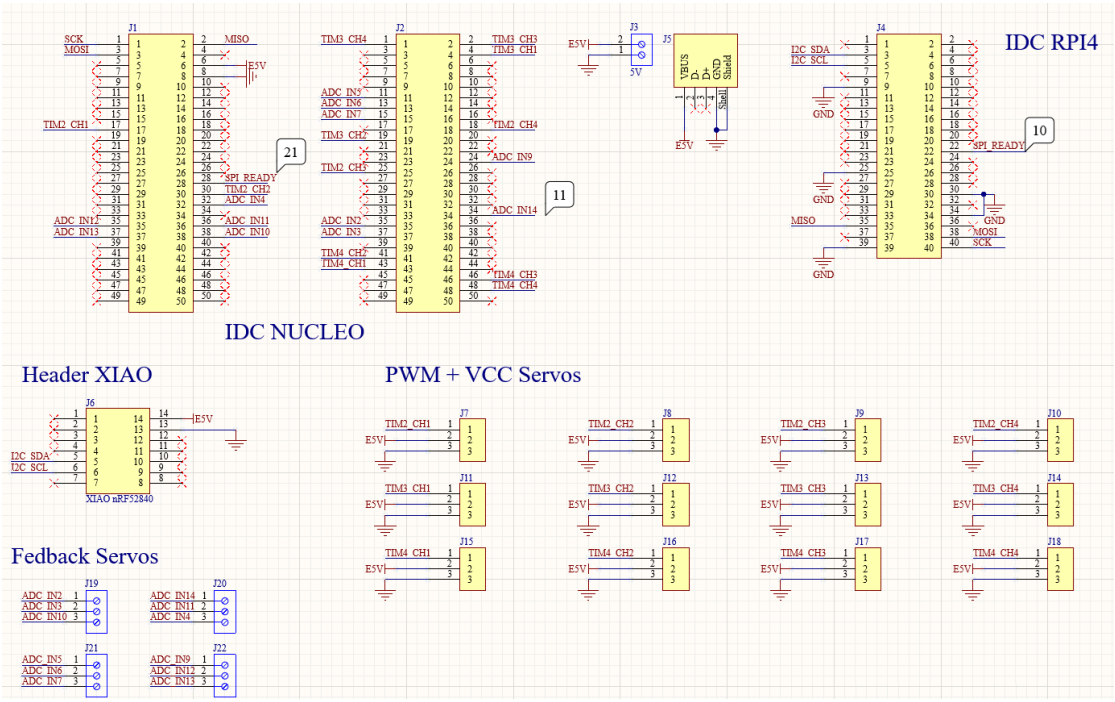


Figura 20: Esquemático Placa Interconexión.

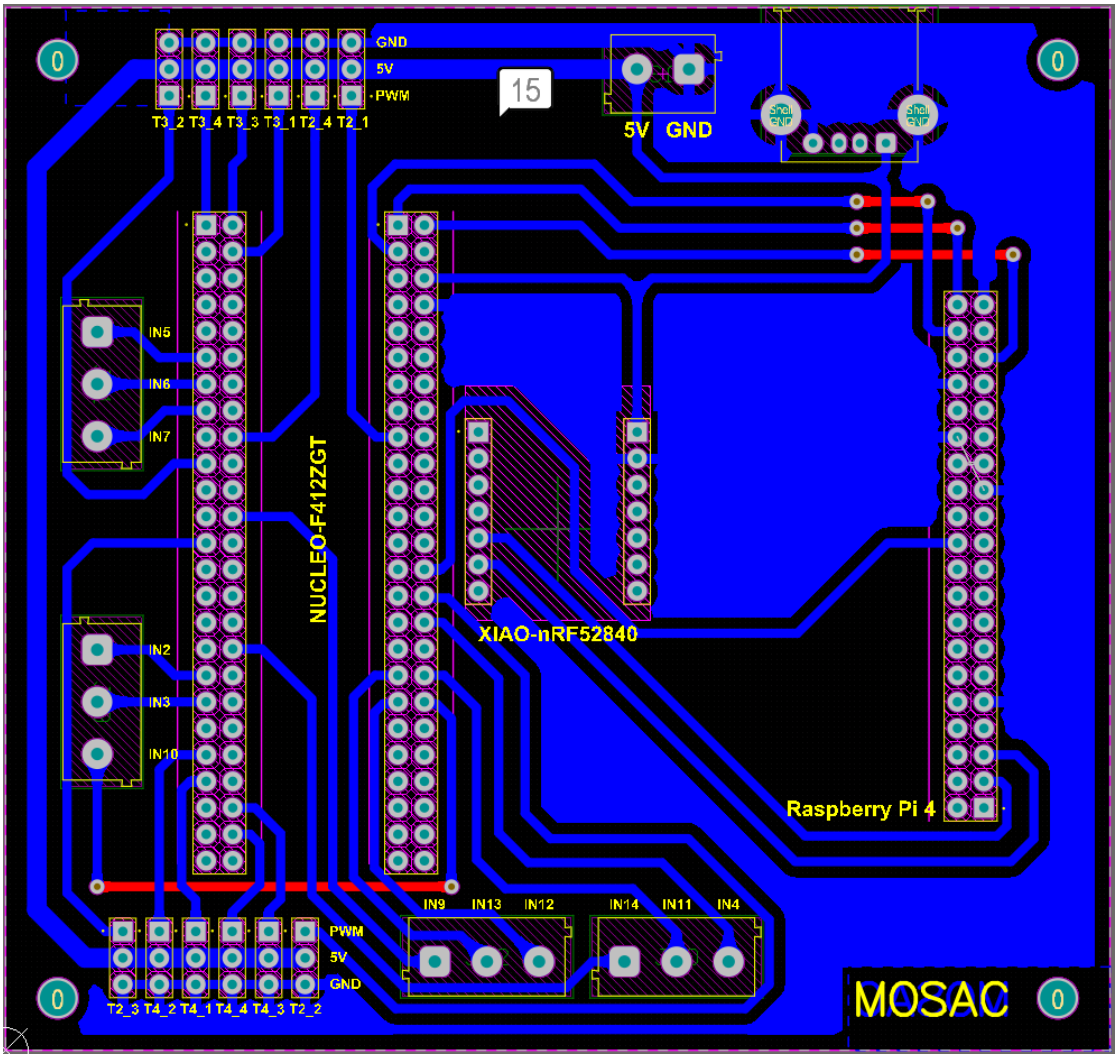


Figura 21: PCB Placa Interconexión.

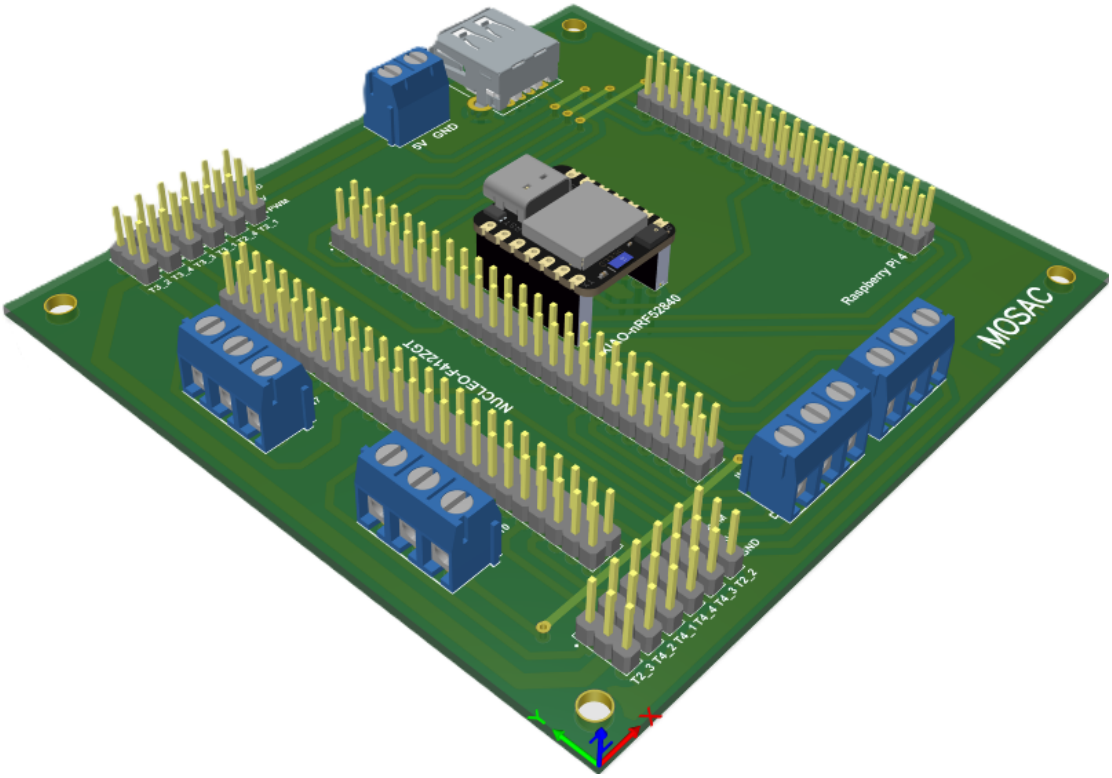


Figura 22: Placa Interconexión.

4.4. Estructura

El robot cuadrúpedo consta de 4 articulaciones con 3 grados de libertad cada una. A estos los llamamos “Body”, “Femur” y “Tibia”. El primero controla la abducción y aducción, y el segundo con el tercero la flexión y extensión de la extremidad. Estos tres grados de libertad son actuados mediante servomotores que permiten lograr los movimientos mencionados con una resolución en grados aceptable. El primer diseño se muestra en la figura 23. Mecánicamente es simple debido a que el eje de cada servo es efectivamente el eje de cada articulación, sin embargo esto trae la gran desventaja de que los servomotores estén ubicados lejos del tronco del cuerpo, y por ende haya un momento de inercia considerable para los servos ubicados en la parte superior de la extremidad. Por otro lado presenta cierta torsión indeseada en las articulaciones dado que el eje del servo está solo en uno de laterales, y se requiere de otra sujeción a lo largo del eje de rotación para mayor robustez. Otra problemática está relacionada al manejo de cables que recorren la extremidad por dentro, y no deben molestar ni estrangularse durante los movimientos.

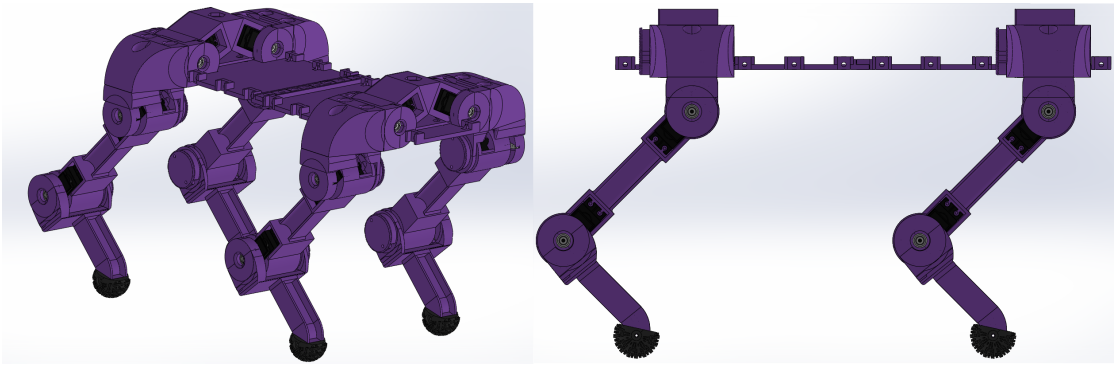


Figura 23: Primer diseño del cuadrúpedo.

Se decidió rediseñar la estructura para solucionar estos problemas (Fig. 24), y además, contemplar la sujeción y protección necesaria para el hardware y la batería (Fig. 25). En la figura 26 se muestra la ubicación del centro de masa en las vistas fundamentales. El mismo se encuentra cerca del centro geométrico del robot, lo cual es lo más deseado.

La segunda versión de la estructura se diseñó en el software SolidWorks y se utilizó una impresora 3D para la fabricación con filamento PLA. Se usó como base un modelo realizado por Baris Alp, publicado en la plataforma GrabCAD [11]. Esta versión cuenta con un rulemán para la unión de la extremidad al cuerpo y dos de cada lado para la articulación inferior, por cada extremidad. Comparando las extremidades de ambas versiones (Fig. 27), se observa como la segunda versión logra minimizar el momento de inercia teniendo todos los servomotores más cerca del cuerpo central, y empleando 4 rótulas que trasladan el movimiento del servo inferior hasta la articulación inferior de la extremidad.

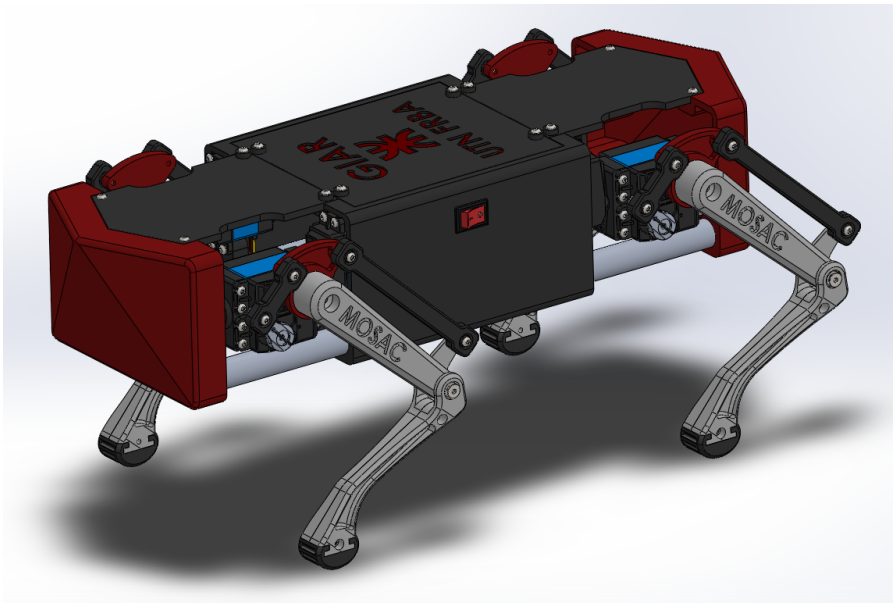


Figura 24: Segundo diseño del cuadrúpedo.

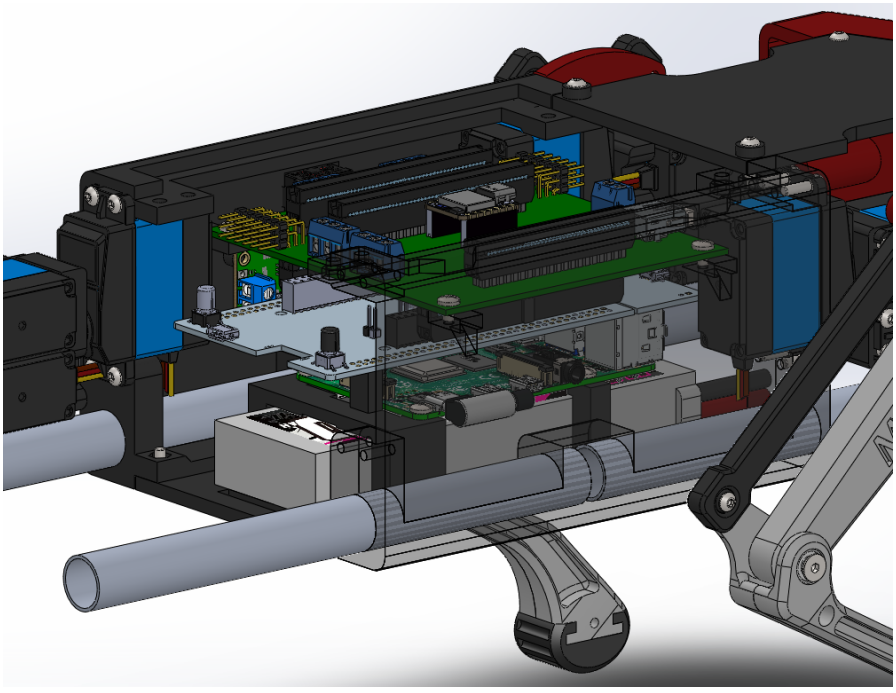


Figura 25: Sujeción del hardware y batería en el segundo diseño.

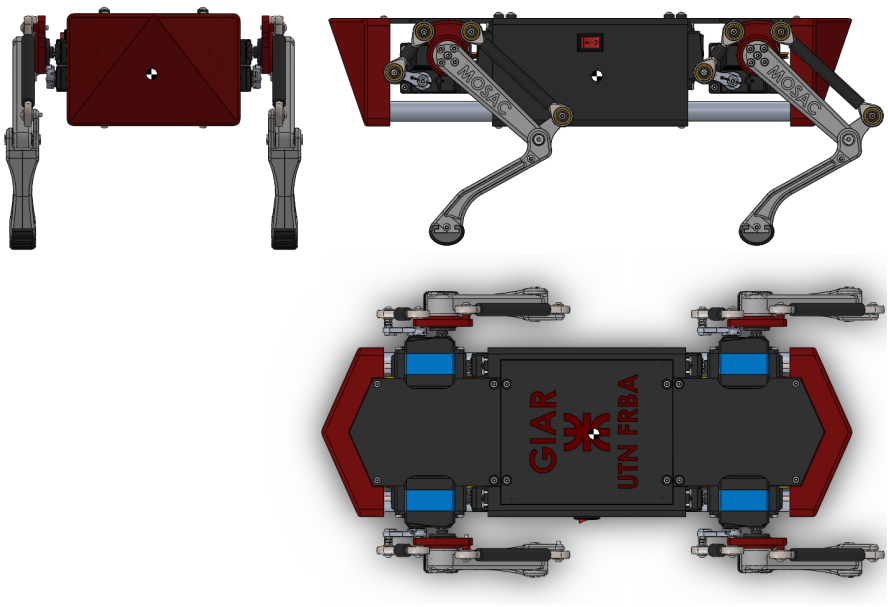


Figura 26: Centro de masa del robot.

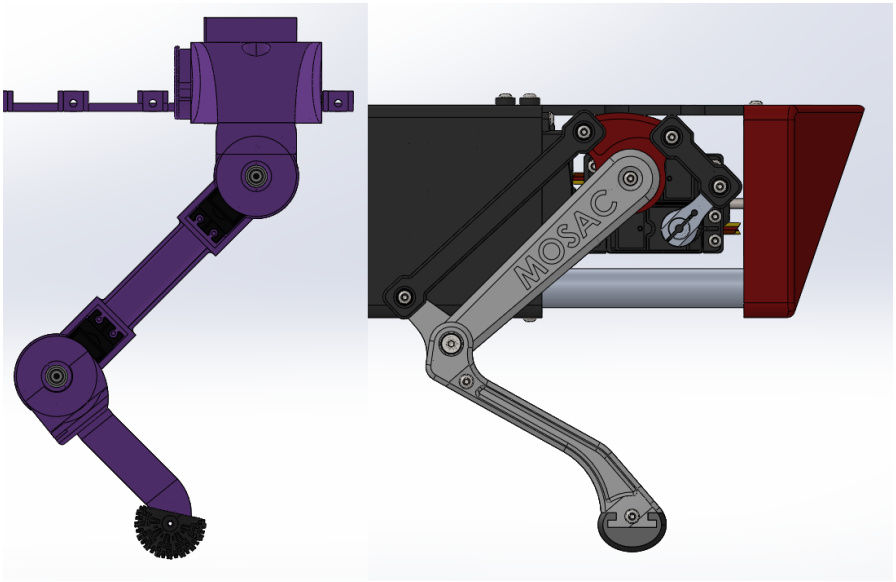


Figura 27: Comparación entre extremidad de primera y segunda versión.

Por último, se realizó una tercera versión de las extremidades para reducir el juego de las partes mecánicas. En primer lugar se reemplazaron las rótulas plásticas diseñadas en la versión 2 por unas metálicas. Estas rótulas se unen por unas varillas plásticas roscadas por dentro, y por simplicidad se denomina “Rótula” al conjunto entero de rotulas con la varilla que las conecta. En cada extremidad hay una “Rótula Grande”

y una “Rótula Chica” que a su vez están conectadas entre sí por una pieza llamada “Shoulder” que permite el movimiento independiente de la Tibia con respecto al Femur (Fig. 28). Por último, el Shoulder que anteriormente tenía cierto juego y fricción con otras piezas plásticas, se rediseñó para que dentro tenga un rulemán de tamaño 6704 que se sujeta al Femur, de esta manera se logra una transmisión más robusta del movimiento del servo hacia la tibia y con menor juego. La impresión 3D de esta parte se realizó en dos fases: la primera consistió en generar la cavidad donde iría el rulemán, se pausó la impresión, se colocó el rodamiento, y luego se reanudó para finalizar la pieza.

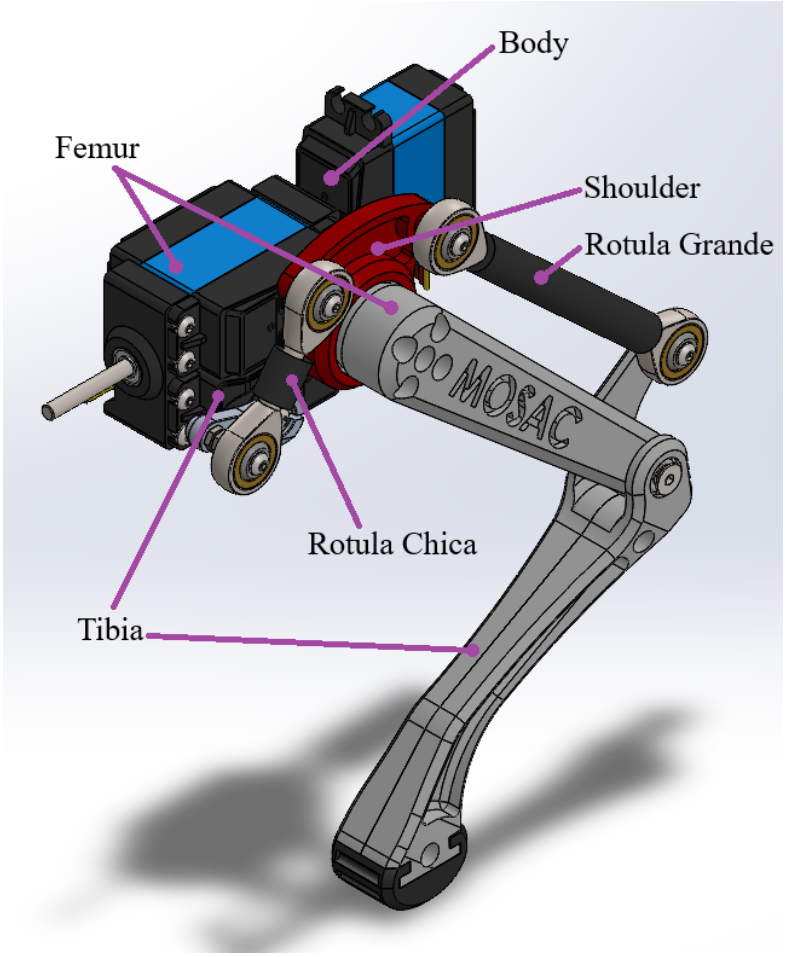


Figura 28: Tercera versión de la extremidad con sus partes.

4.5. Entorno de Simulación - Webots

Como se mencionó anteriormente, el algoritmo de inteligencia artificial puede ser evaluado mediante su ejecución en un robot físico o en un entorno simulado por computadora. La primera opción tiene la ventaja de que el aprendizaje ocurre directamente en el entorno real en el que operará el robot. Sin embargo, en las primeras fases del entrenamiento, es común que el robot realice movimientos bruscos e inestables, lo que podría dañarlo o afectar el funcionamiento de sus actuadores y otros componentes. Además, reiniciar manualmente la posición del robot después de cada episodio de entrenamiento puede resultar tedioso y ralentizar el proceso.

Por otro lado, la simulación permite entrenar al agente sin riesgo de dañar el hardware o las partes mecánicas, y ofrece la posibilidad de ejecutar el algoritmo y la física del entorno a una velocidad superior a la del tiempo real. El inconveniente principal es modelar de la manera más realista posible al robot dentro del simulador, y esto implica no solamente recrear de la manera más fiel toda la estructura sino también, por ejemplo, simular el comportamiento mecánico de los actuadores de acuerdo a las especificaciones de los fabricantes, entre otras cosas.

El simulador utilizado se denomina Webots, desarrollado por Cyberbotics [12]. Permite modelar, programar y probar robots en un entorno virtual. Es ampliamente utilizado en investigación, educación e industria debido a su capacidad para simular con precisión la física y la dinámica de los robots. Utiliza ODE (Open Dynamics Engine) para simular colisiones, fricción, inercia y otros efectos físicos parametrizables. Permite, entre muchas otras opciones, la integración de cámaras, LIDAR, sensores de proximidad, GPS o IMU, y para controlar a los robots utiliza scripts denominados "controladores" que pueden ser programados en C, C++, Python, Java y MATLAB con funciones propias del simulador.

4.5.1. Simulación de estructura del robot

Un aspecto fundamental para simular el robot es tener un modelo lo más realista posible del mismo. Para lograrlo, se generó un archivo URDF (Unified Robot Description Format) a partir del modelo de Solidworks. Este archivo, con formato XML, describe al robot como un conjunto de cuerpos rígidos unidos por todas las articulaciones. Cada cuerpo rígido o "link" a su vez es un conjunto de piezas unidas entre sí, cada una con su masa cargada manualmente a Solidworks en base a mediciones del robot real. Luego, para cada link se calculó la masa total, el centro de masa y la matriz de inercia utilizando las herramientas de Solidworks; también se especificó el eje y sistema de coordenadas donde se ubica la articulación "padre", y se especificaron cuántas articulaciones "hijas" tiene dentro de la jerarquía de links. Para importar el archivo URDF a Webots primero se debe transformar a formato "proto", para lo cual se utilizó un paquete de python también suministrado Cyberbotics, creadora de Webots, llamado "urdf2webots". Una vez en Webots (Fig. 29), el modelo fue editado para agregar las constantes de fricción necesarias, y las conexiones articulares que no responden a la jerarquía padre e hijo sino

que completan un loop mecánico en cada extremidad.

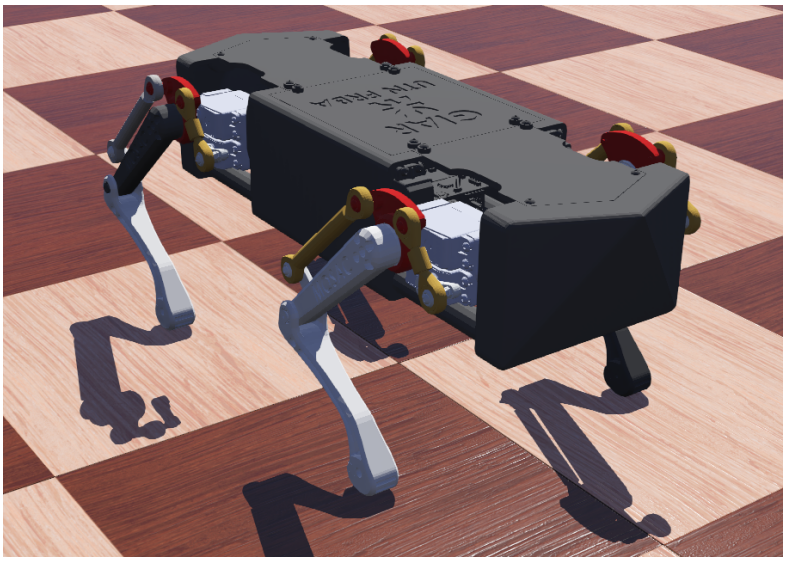


Figura 29: Modelo realista importado a Webots.

4.5.2. Simulación de servomotores del robot

Otro aspecto importante a recrear es el movimiento que tienen los actuadores. En otras palabras, ante las mismas condiciones iniciales, ángulo target, fricción y momento de inercia conectado al eje, la posición angular en el simulador debe variar en el tiempo de la misma forma que en la realidad. Esa prueba es la que se utilizó para comparar los motores reales con los simulados.

Según la documentación de Webots los actuadores por defecto utilizan un PID interno configurable. Esto sugiere que un primer intento para acercar la simulación a la realidad podría ser modificar los parámetros del controlador de Webots hasta que se asemejen las curvas de posición angular en el tiempo. Sin embargo, tras varios intentos fallidos y mayor investigación, se llegó a la conclusión de que la razón por la que nunca se iba a lograr un resultado aceptable es que el motor que se está controlando con el PID funciona fundamentalmente distinto a los motores de continua que tiene el robot real: la documentación del simulador ODE que utiliza Webots aclara que los actuadores implementados tienen el objetivo de alcanzar una velocidad target al final del paso de simulación, ejerciendo la cantidad exacta de torque para lograrlo. Este valor de torque se averigua "efectivamente viendo un paso de simulación en el futuro" para poder contrarrestar cualquier torque externo que pueda aparecer. La razón por la que se simulan de esta forma a pesar de ser irrealistas, es para lograr motores robustos y estables con los que simular. Sin embargo, en comparación con un motor de continua no hay ninguna similitud: la velocidad no se establece sino que es el producto de la interacción del torque producido contra el momento de inercia, rozamiento viscoso y constante, y cualquier otro

torque externo que pueda surgir en el entorno. Por esta razón, se decidió no utilizar los actuadores por defecto sino modelar completamente los servomotores, tanto el motor de continua como el controlador PID interno.

La figura 30 muestra el modelo de un motor de continua.

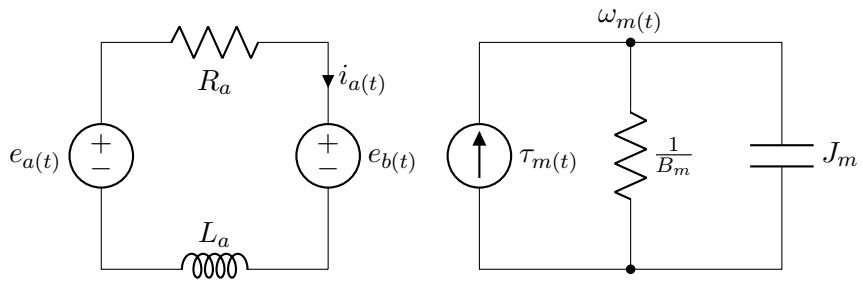


Figura 30

Las ecuaciones que derivan de dicho modelo, utilizando la transformada de Laplace, son:

$$E_{b(S)} = K_b \; \Omega_{m(S)}$$
$$\tau_m = K_t \; I_{a(S)}$$
$$I_{a(S)} = \frac{E_{a(S)} - E_{b(S)}}{SL_a + R_a}$$
$$\Omega_{m(S)} = \frac{T_{m(S)}}{SJ_m + B_m}$$

Sin embargo, es necesario también incorporar el sistema de engranajes conectado al eje del motor de continua, junto con sus momentos de inercia y rozamientos viscosos. Estos se contemplan circuitalmente en la figura 31. Tal como se presenta el circuito, sería necesario conocer las relaciones de transformación de todos los pares de engranajes conectados, junto con los momentos de inercia y rozamientos viscosos individuales (η, B, J). Para simplificar los ensayos, se reflejan todos los componentes hacia el subcircuito donde se encuentra el eje del servo con la velocidad y torque final $(\omega_{out(t)}, \tau_{out(t)})$, y se agrupan todos los rozamientos y momentos de inercia reflejados con B_4 y J_4 respectivamente. Esto está representado en la figura 32.

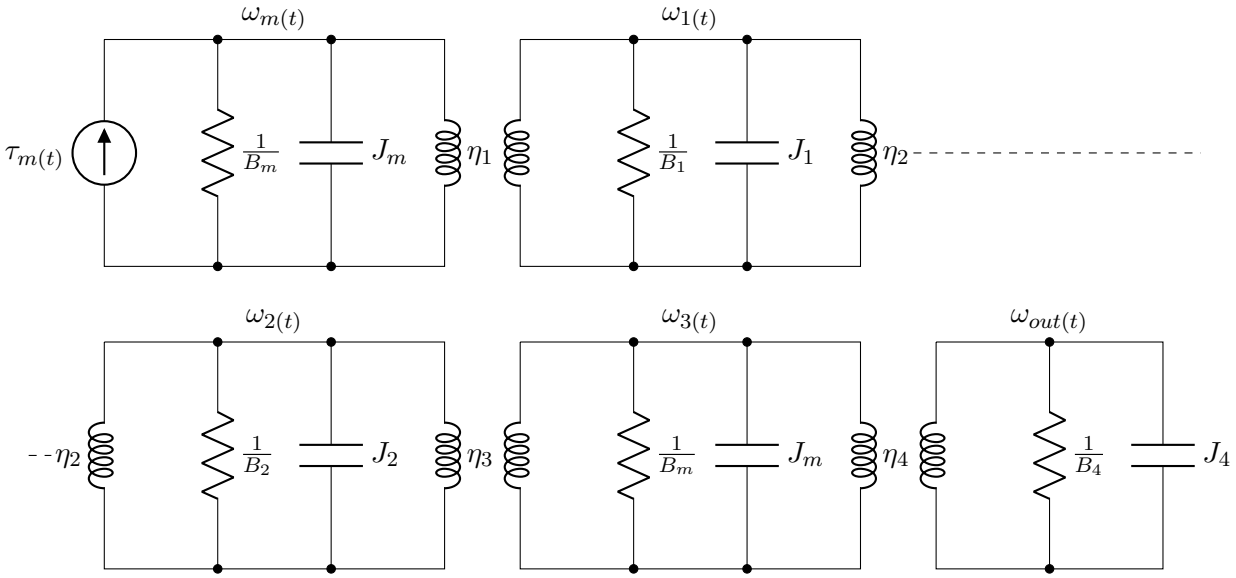


Figura 31: Mecánica de motor de continua con tren de engranajes.

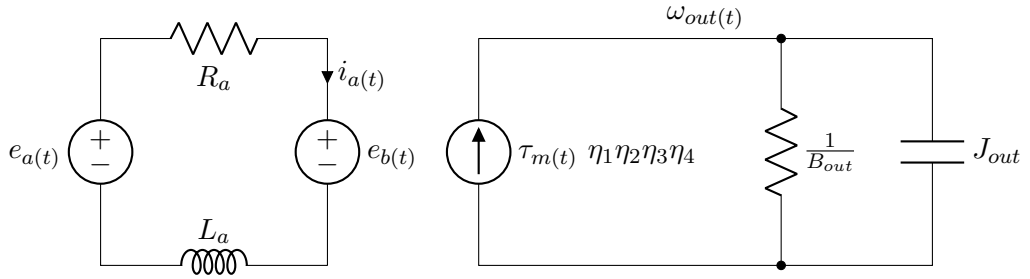


Figura 32: Motor de continua completo con tren de engranajes simplificado.

A su vez, se redefinen las constantes K_b y K_t del motor con el objetivo ignorar las relaciones de transformación (η) y de trabajar directamente con $\omega_{out(t)}$ en lugar de $\omega_{m(t)}$ que es mucho mayor debido a la relación de transformación, y por ende más difícil de medir:

$$e_{b(t)} = K_b\ \omega_{m(t)} = K_b\ \eta_1\eta_2\eta_3\eta_4\ \omega_{out(t)} = K_b'\ \omega_{out(t)}$$
$$\tau_{out(t)} = \tau_{m(t)}\ \eta_1\eta_2\eta_3\eta_4 = K_t\ i_{a(t)}\ \eta_1\eta_2\eta_3\eta_4 = K_t'\ i_{a(t)}$$

Como $K_b = K_t$, se define $K_m \equiv K_b' = K_t'$. Con este modelo simplificado, los parámetros a obtener del motor son $R_a, L_a, K_m, B_{out}, J_{out}$:

1. **Resistencia de armadura R_a:** Para este ensayo, se tuvo que separar el motor de continua del resto del servomotor, luego se bloqueó el rotor para que la fuerza contra-electromotriz sea nula $e_{b(t)} = K_b\ \omega_{(t)} = 0$, se aplicó una tensión constante

y reducida, y se midió la corriente en circulación. Se observó que el tiempo de establecimiento de la corriente es despreciable en comparación con los pasos de simulación en el orden de milisegundos, por lo que se puede aproximar $L_a = 0H$.

2. **Constante del servomotor K_m:** Se conectó el motor al tren de engranajes (sin conectar todavía la electrónica que controla el servomotor), y se partió de la ecuación de malla del circuito eléctrico del motor:

$$e_{a(t)} - R_a i_{a(t)} - K_m \omega_{out(t)} - L_a \frac{di_{a(t)}}{dt} = 0$$

Por más de que L_a sea despreciable, su efecto es completamente nulo al aplicar una tensión constante $e_{a(t)} = E_a$ y realizar las mediciones en estado estacionario $t \to \infty$, resultando en:

$$K_m = \frac{E_a - R_a I_a}{\omega_{out}}$$

Para conocer la velocidad constante, se midió el tiempo $T_{vueltas}$ que le tomó al eje del servo completar N cantidad de vueltas:

$$\omega_{out} = \frac{2\pi N}{T_{vueltas}}$$

3. **Momento de inercia total del servomotor J_{out}:** Al aplicar una tensión constante en la armadura, y esperar a que el motor de C.C. junto con el tren de engranajes llegue al estado estacionario, el torque generado por el motor contrarresta al torque externo de carga τ_{load}, que está compuesto por rozamiento viscoso e incluso por un rozamiento constante τ_f si es que existe (el momento de inercia no tiene efecto porque la velocidad angular es constante):

$$\tau_{out} = \tau_{load} = B_{out}\ \omega_{out} + \tau_f \tag{1}$$

Si en esta situación, se desconecta la tensión de armadura (supongamos en el instante $t = 0$), y se deja un circuito abierto que garantiza $i_a = \tau_{out} = 0$, entonces el torque de carga es el único responsable de reducir la velocidad del motor:

$$\tau_{load} = B_{out}\ \omega_{out} + \tau_f = -J_{out} \frac{d\omega_{out(0^+)}}{dt}$$

A su vez, en el instante inmediato luego de desconectar la fuente ($t = 0^+$) la velocidad permanece igual, y por ende τ_{load} tampoco varía. Esto hace que el torque de carga, igual al torque que ejercía el motor en el instante anterior a la desconexión, se pueda igualar a la desaceleración del motor:

$$\tau_{load} = \tau_{out} = K_m\ i_{a(0^-)} = -J_{out} \frac{d\omega_{out(0^+)}}{dt} \tag{2}$$

Midiendo la corriente en estado estacionario $i_{a(0^-)}$ y la pendiente con la que se reduce la velocidad en el instante de desconexión $-\frac{d\omega_{out(0^+)}}{dt}$, es posible despejar de

la ecuación 2 el momento de inercia del eje del motor de continua combinado con el tren de engranajes completo J_{out}. Para medir la pendiente de la velocidad, se midió la tensión de armadura luego de la desconexión, la cual corresponde directamente a $e_{b(t)} = K_m\ \omega_{out(t)}$. Por ende:

$$J_{out} = \frac{K_m\ i_{a(0^-)}}{\frac{1}{K_m}\frac{de_{b(0^+)}}{dt}} = \frac{K_m^2\ i_{a(0^-)}}{\frac{de_{b(0^+)}}{dt}}$$

4. **Rozamiento viscoso total del servomotor B_{out}:** De la ecuación 1, si asumimos que el rozamiento constante τ_f es despreciable se puede obtener B_{out} con los datos del ensayo anterior en estado estacionario:

$$B_{out} = \frac{\tau_{out}}{\omega_{out}} = \frac{K_m\ I_a}{\omega_{out}}$$

Sin embargo, al repetir el ensayo con distintas tensiones de armadura (y por ende distintos valores de I_a y ω_{out}), la constante de rozamiento viscoso difería demasiado. Esto quiere decir que τ_f no es despreciable. Por esta razón se tomaron 2 mediciones, para formar un sistema de 2 ecuaciones y poder obtener B_{out} y τ_f simultáneamente:

$$\begin{cases} K_m\ I_{a1} = B_{out}\ \omega_{out1} + \tau_f \\ K_m\ I_{a2} = B_{out}\ \omega_{out2} + \tau_f \end{cases} \Longrightarrow B_{out} = \frac{K_m(I_{a1} - I_{a2})}{\omega_{out1} - \omega_{out2}}$$

Una vez obtenido B_{out}, se puede despejar fácilmente τ_f de cualquiera de las dos ecuaciones anteriores.

Los resultados de los ensayos se muestran en la tabla 3.

Parámetro	Símbolo	Valor	Unidad
Resistencia de armadura	R_a	1,45	Ω
Constante del motor	K_m	0,98	$\frac{N \cdot m}{A} = V \cdot s$
Momento de inercia	J_{out}	$6{,}47 \cdot 10^{-3}$	$N \cdot m \cdot s^2$
Rozamiento viscoso	B_{out}	$4{,}23 \cdot 10^{-3}$	$N \cdot m \cdot s$
Rozamiento constante	τ_f	$32{,}62 \cdot 10^{-3}$	$N \cdot m$

Tabla 3: Parámetros medidos del conjunto motor de C.C. y tren de engranajes.

Luego de realizar los ensayos, los valores obtenidos fueron validados a partir de una simulación en Webots, colocando un cubo con la masa suficiente como para igualar el momento de inercia total obtenido en el rotor J_{out} (Fig. 33). En los ensayos con el motor real, al aplicar una tensión constante de $8{,}4V$ en la armadura, se consumía una corriente de armadura de aproximadamente $70mA$, y se alcanzaba una velocidad constante de $8{,}49\frac{rad}{s}$.

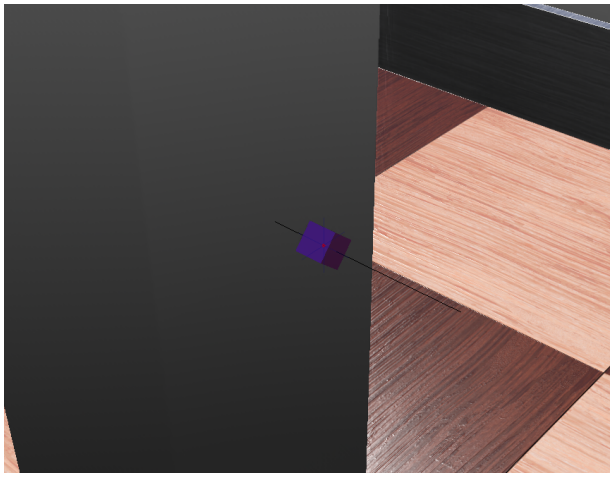


Figura 33: Simulación del motor de C.C. con J_{out}.

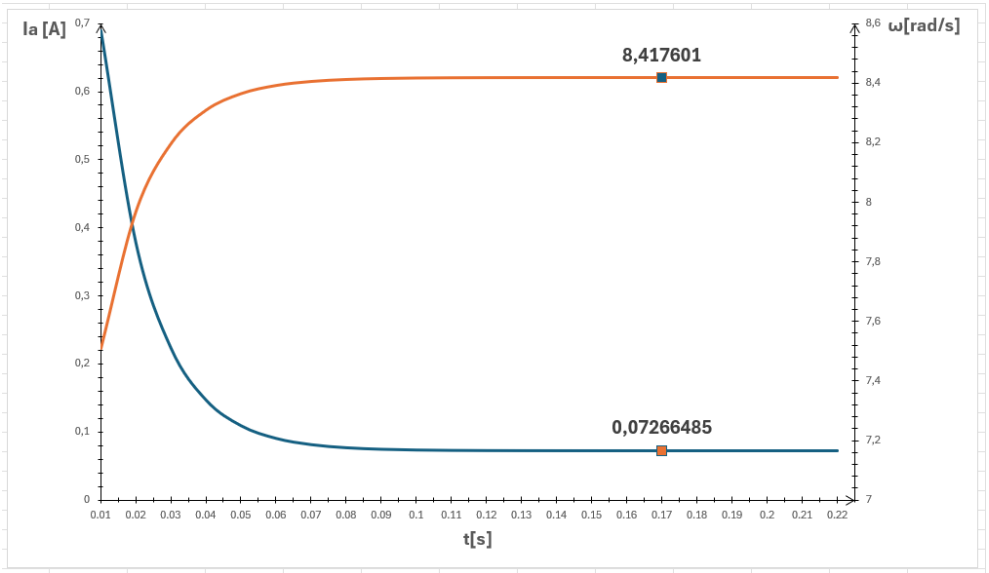


Figura 34: Corriente tomada por el motor de C.C. del servo y velocidad angular en estado estacionario del rotor.

Como se puede observar en la figura 34, el valor final de corriente y velocidad angular en estado estacionario es de aproximadamente $0{,}073mA$ y $8{,}42\frac{rad}{s}$ respectivamente, lo cual corrobora que el motor de corriente continua que tiene el servomotor es simulado correctamente.

El siguiente paso es obtener las constantes del PID interno que controla el servomotor. Para ello, se utiliza una herramienta de MATLAB - Simulink llamada “Parameter

Estimation" que permite estimar parámetros de un sistema comparando la salida de un sistema simulado en Simulink, con la salida del sistema real, a partir de una misma señal de excitación. En este caso, la señal de salida es la posición angular del servomotor. Se podría obtener del potenciómetro interno del servomotor y luego filtrarse y convertirse a ángulo, pero luego de una serie de pruebas se vio que no tenía la precisión necesaria como para compararse con la señal de un sistema simulado sin interferencias de ruido eléctrico.

Por lo tanto, se optó por utilizar un encoder magnético **AS5600** de AMS OSRAM para obtener la posición angular del rotor, el cual posee una resolución angular de 0.087° en un rango de 360°. Para utilizarlo, se debe colocar un imán en el eje del servomotor que esté alineado con la capsula del integrado. La principal dificultad de este tipo de sensor es alinear correctamente el imán con el IC y dejar la separación vertical necesaria entre ambos (Fig. 35).

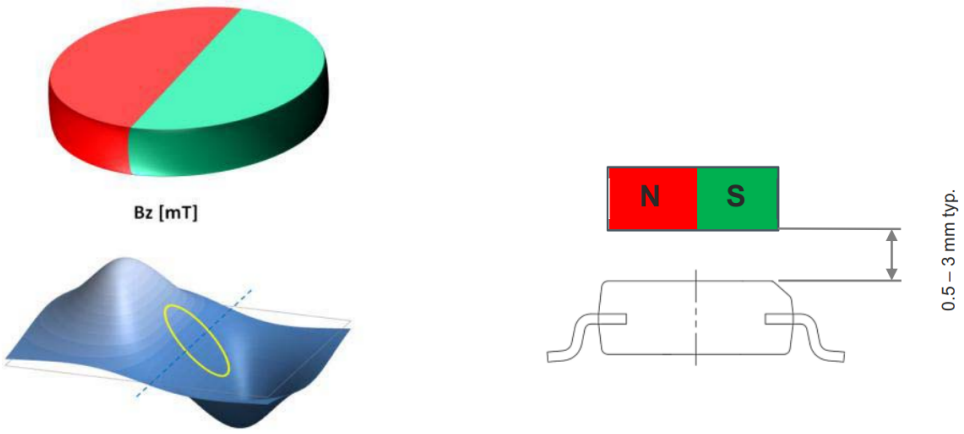


Figura 35: Alineación del imán y separación típica con el AS5600. El IC debe estar alineado de forma tal que se encuentre en la zona lineal del gradiente de campo magnético.

Por lo tanto, se diseñó una estructura impresa en 3D para sujetar un servomotor y alinearlo correctamente con el encoder, para tomar mediciones (Fig. 36).

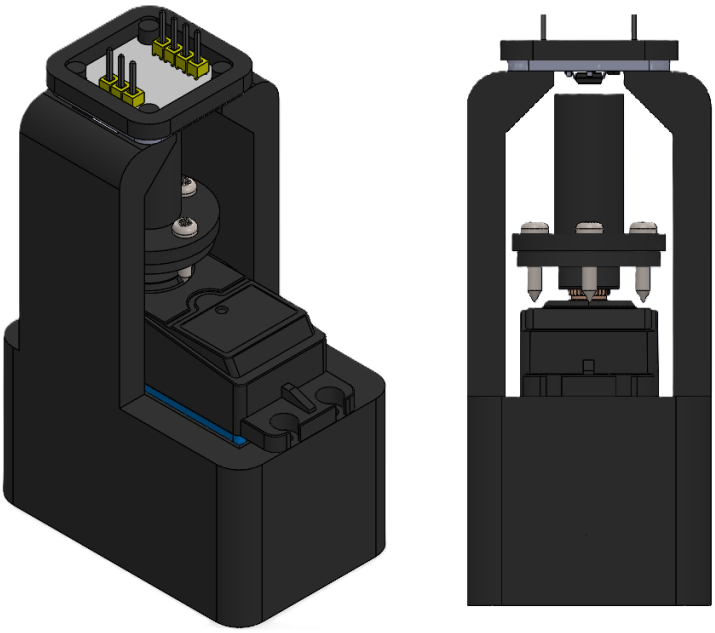


Figura 36: Soporte para alinear imán, rotor y AS5600.

Se utilizó una NUCLEOF401RE para comunicarse a través de I2C con el sensor y obtener muestras de la posición angular del rotor. Los resultados se muestran en la figura 37.

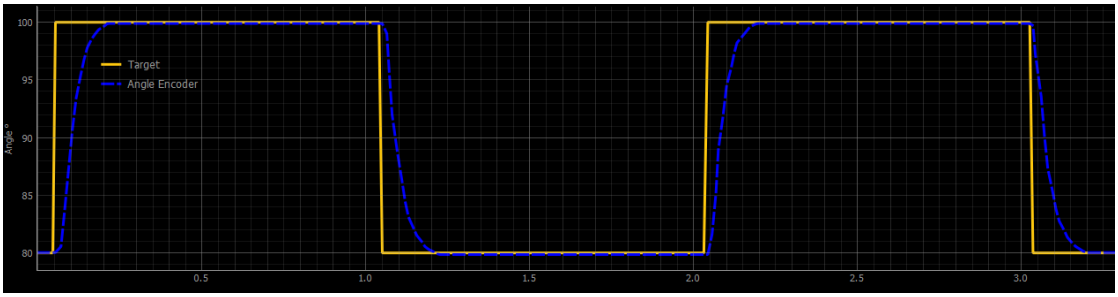


Figura 37: En naranja, el target de posición angular y en azul punteado, la posición angular del rotor obtenida a partir del encoder magnético.

Ya con una señal acorde para realizar la estimación de parámetros, se procedió a realizar un diagrama en bloques en Simulink del sistema para utilizar la herramienta mencionada anteriormente (Fig. 38):

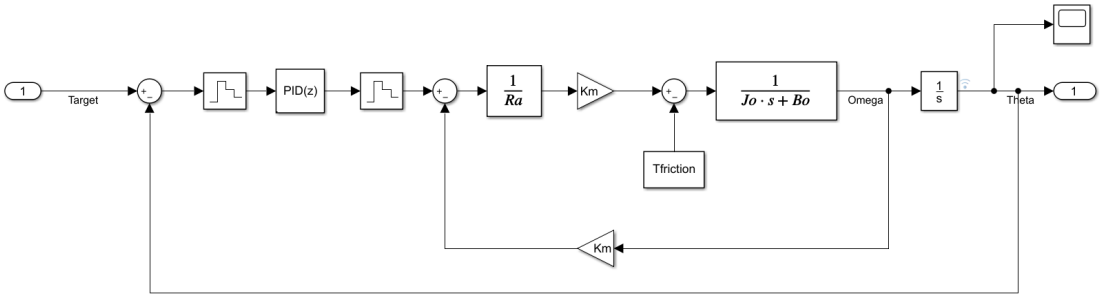


Figura 38: Diagrama en bloques del sistema que representa al servomotor, con su controlador PID discreto.

Luego de una serie de iteraciones, la herramienta convergió en $K_P = 18{,}8470$, $K_I = 2{,}0231$ y $K_D = 0{,}02354$.

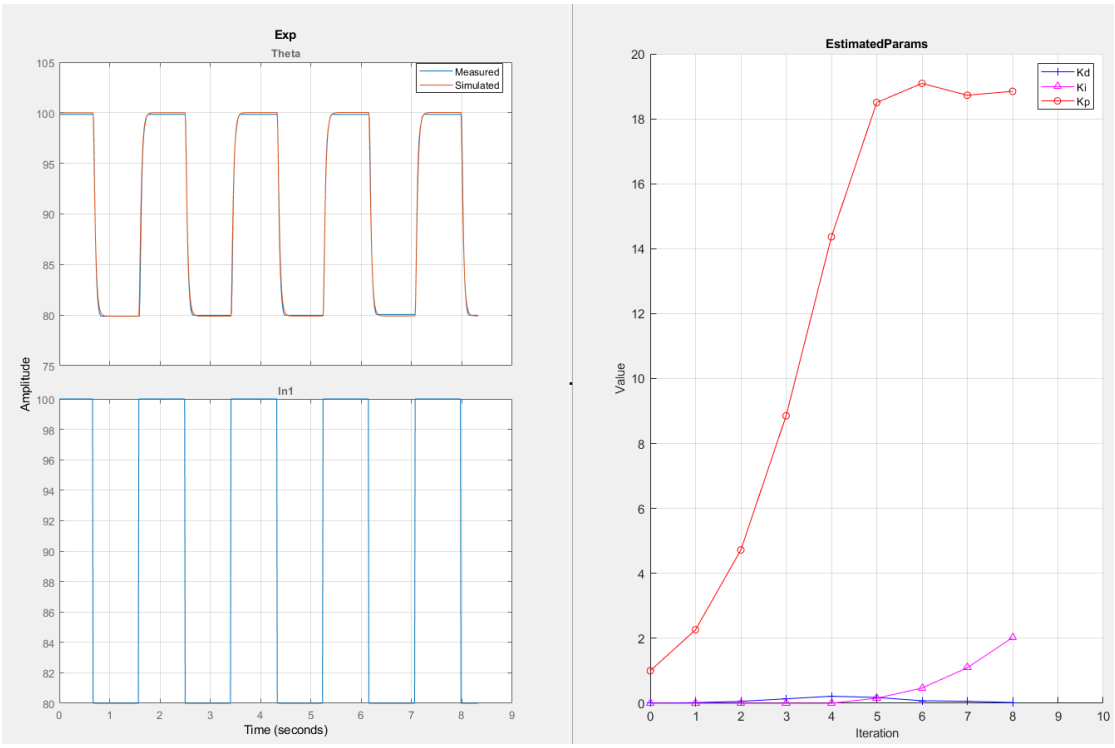


Figura 39: Gráfico arriba a la izquierda: Salida simulada y medida. Gráfico derecha: Iteraciones de K_P, K_I y K_D. Gráfico abajo a la izquierda: Entrada del sistema.

En la figura 40 se muestra la comparación de la curva real con la simulada en Webots, habiendo agregado el PID con las constantes estimadas previamente.

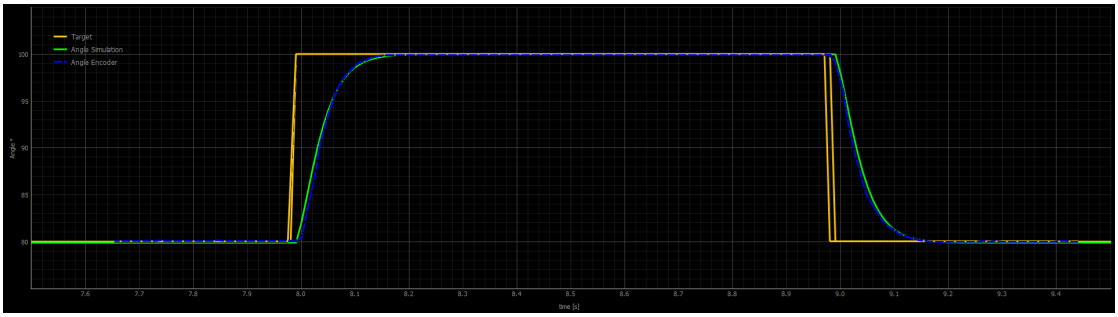


Figura 40: En naranja el target, en verde la posición angular del servomotor simulado, y en azul punteado la posición angular real.

Como se puede observar la simulación logra replicar el comportamiento del servomotor real con errores despreciables. Este resultado es sumamente importante, ya que es crucial no solo que se llegue al mismo valor final, sino la forma en la se hace, es decir, la rapidez de movimiento del servomotor respecto al valor target.

El código utilizado para simular el servomotor en Webots se muestra a continuación:

```
def computeTorques(target_servo):
    for i in range(len(joint)):
      internal_angle_servo[i] = joint_sensor[i].getValue()
      joint_angular_velocity_servo[i] = (internal_angle_servo[i] -
      ↪  prev_joint_angular_position_servo[i]) / (timestep * 1e-3)
      prev_joint_angular_position_servo[i] = internal_angle_servo[i]

    # PID control
    error_servo = target_servo - internal_angle_servo
    delta_error_servo = (error_servo - prev_error_servo) / (timestep * 1e-3)
    acum_error_servo += error_servo * timestep * 1e-3
    prev_error_servo = error_servo

    # Voltage and current calculations
    VA = KP_servo * error_servo + KD_servo * delta_error_servo + KI_SERVO *
    ↪  acum_error_servo
    VA = np.clip(VA, -8.4, 8.4)

    ####### Not used, only useful to know real back emf, current and torque #######
    Eb = joint_angular_velocity_servo * KM
    Ia = (VA - Eb) / RA
    joint_torque = Ia * KM
    ################################################################################

    # Apply torque to joints
    for i in range(len(joint)):
      #Make sure the constant friction always opposes the motor's torque
      if VA[i] < 0:
        friction_direction = -1
      else:
        friction_direction = +1
```

```
30
31          joint[i].setTorque(VA[i]*KM/RA - friction_direction*TF)
32          #After computing the DC Motor Control, add noise for the rest of the controller
33          #f_joint_angle[i] = f_joint_angle[i]+ np.random.normal(0,0.5,None)  #!(Optional
            ↪  noise)
```

Se puede observar que el PID interno regula la tensión de armadura que luego recibe el motor de C.C. tal como sucede en el servomotor real. La diferencia es que mientras en la simulación la tensión es analógica, tomando valores de -8.4V a 8.4V (tensión máxima de la batería), en la realidad se modula el valor medio de la tensión de armadura con un PWM y un puente H (para aplicar tensión positiva o negativa).

Por otro lado, fue necesario modelar la fuerza contra-electromotriz $e_{b(t)}$ como un rozamiento viscoso adicional en lugar de simplemente obtener el valor en el script y restarlo a la tensión de armadura, para hacer que la simulación fuese más estable:

$$\begin{aligned} \tau_{out(t)} &= \frac{e_{a(t)} - e_{b(t)}}{R_a} K_m \\ &= \frac{e_{a(t)}}{R_a} K_m - \omega_{out(t)} \frac{K_m^2}{R_a} \\ \therefore \frac{K_m^2}{R_a} \equiv B_\omega &\Longrightarrow B_{total} = B_{out} + B_\omega \end{aligned}$$

4.5.3. Simulación del hardware del robot

Si bien se puede obtener la inclinación directamente del motor de física del entorno de simulación, decidimos utilizar modelos simulados del acelerómetro y el giróscopo, ubicándolos en la misma posición donde se encuentra la IMU en el robot real, agregando a las señales de los sensores un ruido gaussiano con el desvío estándar informado por la hoja de datos del LSM6DS3TR-C. Luego, en el script del controlador de la simulación, se ejecuta el mismo algoritmo del filtro de Kalman utilizado en la IMU real, con los mismos parámetros. De esta forma se obtienen señales de Pitch y Roll similares en cuanto a ruido y desfasaje temporal.

Por otro lado, también se implementan los mismos algoritmos presentes en la NUCLEO: la máquina de estados supervisora y el controlador PI adicional al PID interno de los servomotores, ambos detallados en la sección 4.2.3. En este caso no se tuvo que estimar ningún parámetro, sino simplemente utilizar los mismos valores que los usados en el microcontrolador. Lo único a tener en cuenta es que el control PI se debe ejecutar a la misma frecuencia (100Hz), y que la salida del PI real son los microsegundos del ancho de pulso de la señal PWM que controla los servomotores mientras que en Webots los servomotores directamente utilizan un valor target en grados. Para realizar la conversión se utilizó una constante K_TON que vincula linealmente el ancho de pulso en microsegundos con la posición angular en grados. Esta constante se obtuvo de los ensayos de calibración de los servomotores.

```python
def PIDControl():
    for i in range(len(joint)):
    f_joint_angle[i] = delay_buffer[i][-1]

    error = target_joint - f_joint_angle
    delta_error = (error - prev_error) / (timestep * 1e-3)
    acum_error += error * timestep * 1e-3
    prev_error = error

    control_signal = np.round(KP * error + KD * delta_error + KI * acum_error) / K_TON

    pid_out = pid_out + control_signal

    out_of_max_range = pid_out > jointUpperLimit

    out_of_min_range = pid_out < jointLowerLimit

    #If any joint steps out of the range, and the PID
    pid_out = pid_out * (1 - (out_of_max_range+out_of_min_range)) + jointUpperLimit *
    ↪  out_of_max_range + jointLowerLimit * out_of_min_range

    pid_sample = 1
```

Para validar el control PI, se comparó el movimiento del robot real y el simulado apoyados sobre el suelo (con carga), con la misma señal target aplicada en las tibias. En la figura 41 se puede observar una coincidencia de las curvas en los momentos de transición, e incluso el mismo error al target por parte de ambos servomotores.

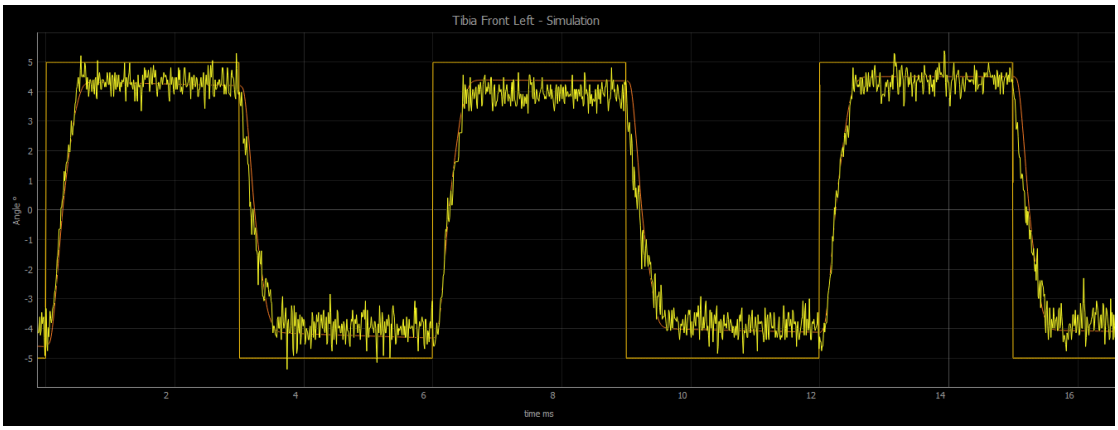


Figura 41: En naranja claro el target, en naranja oscuro posición angular de servomotor simulado, en amarillo posición angular real.

5. Resultados

A pesar de tomar varias medidas para acercar el modelo de simulación lo más posible a la realidad, siguen habiendo diferencias que no permiten transferir el rendimiento que

tiene el agente durante el entrenamiento simulado al robot real. Las razones con mayor probabilidad de afectar los resultados son:

1. **Centros de masa:** A pesar de haber calculado los centros de masa para cada cuerpo rígido unido de la estructura, parece haber una mínima desviación con el modelo real. Esta desviación es difícil de determinar, pero es notable al realizar movimientos controlados: al levantar dos patas cruzadas (por ejemplo delantera derecha y trasera izquierda), en la simulación el cuerpo permanece estable por algunos segundos, mientras que en el robot real el cuerpo se inclina inmediatamente hacia adelante, solo dejando la pata trasera en el aire.

2. **Contacto de patas con el suelo:** Hay varios parámetros que modifican la forma en la que interactúa el robot con el suelo, relacionados a distintos tipos de fricción, rebote, y fuerzas que determinan qué tanto se hunde el robot en el suelo: coulombFriction, frictionRotation, rollingFriction, CFM y ERP. La descripción de cada parámetro se puede encontrar en la documentación de Webots [12], en la sección de *"Nodes and API Functions/contactProperties"*. Estos parámetros no solo determinan cuánto se deslizan las patas en contacto con el suelo, sino que también afectan la estabilidad del robot al levantarlas. Se realizaron múltiples pruebas con distintas combinaciones de valores, usando tablas de fricción estática para distintos materiales de referencia, pero no se encontró todavía una combinación óptima: en algunos movimientos una pata que debe deslizarse en la simulación se traba en la caminata real, y viceversa.

3. **Frecuencia de simulación de servomotores:** Si bien los actuadores son más realistas teniendo el motor de continua y el control PID con las constantes estimadas, la frecuencia máxima a la que se puede ejecutar el control PID y el motor de continua es de 100Hz a fines prácticos (podría incrementarse hasta 1000Hz pero la simulación se vuelve demasiado lenta). El control de los servos reales se ejecuta en el orden de los kHz, y el motor de continua es analógico, por lo que discretizarlo temporalmente con pasos de 10ms representa una desviación incluso mayor de la realidad.

A continuación se muestra la caminata obtenida luego del entrenamiento en el entorno simulado. Durante esta prueba se le indica al agente que se mantenga derecho durante los primeros 80 pasos, y luego cada 20 pasos rote con distintos ángulos objetivo: 6° hacia la derecha, 5° hacia la izquierda, 0° nuevamente, 10° hacia la derecha y finalmente 10° hacia la izquierda.

A continuación se muestran los resultados del testeo en simulación de la caminata en la figura 42, 43 y 44:

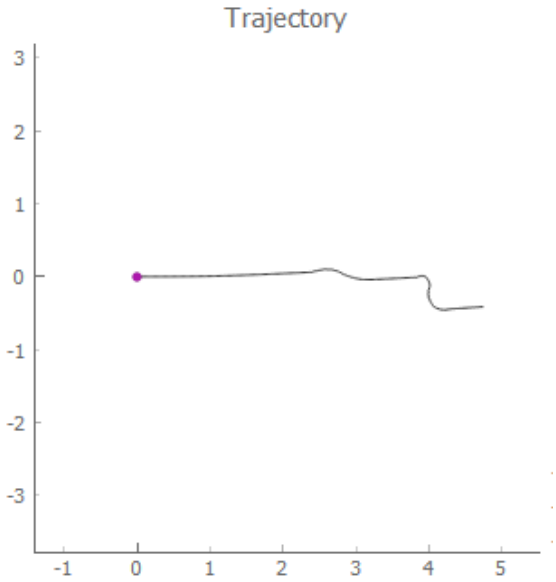


Figura 42: Trayectoria realizada durante el testeo en la simulación.

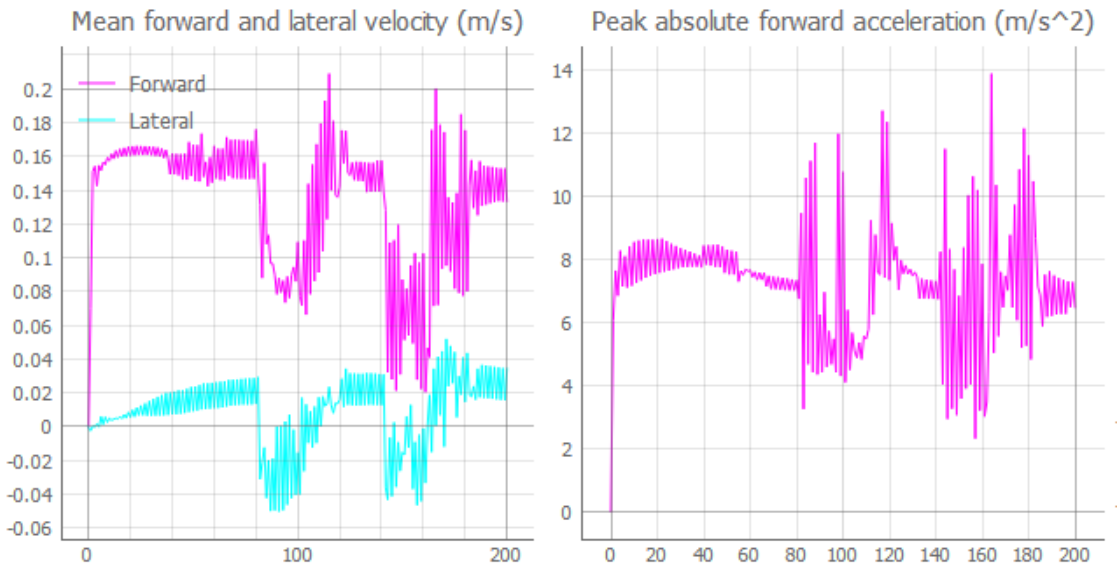


Figura 43: Izquierda: velocidad media lateral y hacia adelante del robot. Derecha: aceleración máxima hacia adelante del robot.

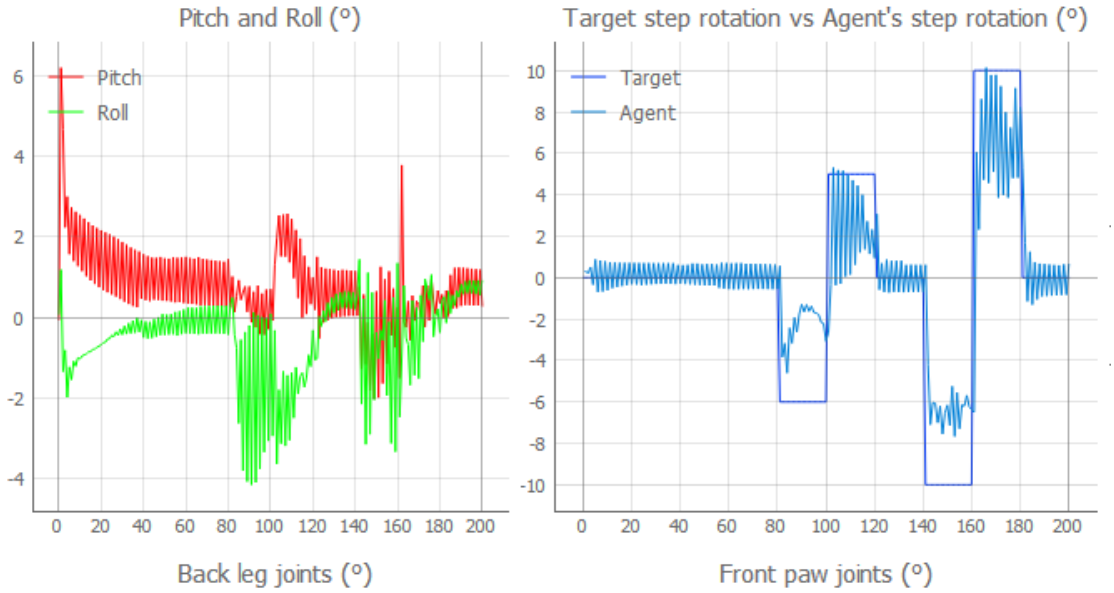


Figura 44: Izquierda: inclinación en pitch y roll del robot. Derecha: rotación del agente con su respectivo target.

Posteriormente se muestra dicha caminata en el robot real, tanto en el aire como apoyado sobre el suelo.

Como se puede observar, la política parece tener problemas para replicar la misma estabilidad corporal y contacto con el suelo en el robot real.

Por último, para mostrar la funcionalidad mecánica del robot se muestra una caminata programada. Para poder realizarla, se tuvieron que desarrollar las ecuaciones de cinemática inversa que permiten averiguar las posiciones angulares de los servomotores en función de las coordenadas deseadas para cada pata. Nótese que en esta caminata, para garantizar la estabilidad del robot, solo se levanta una extremidad a la vez, y antes de levantarla se inclina el cuerpo del robot para desplazar el centro de masa hacia la esquina opuesta.

6. Conclusiones

Si bien no se pudo alcanzar el mismo comportamiento de la inteligencia artificial en el robot real, se considera que los resultados obtenidos son satisfactorios teniendo en cuenta el tiempo disponible y el desafío que representa en el ámbito del aprendizaje por refuerzo minimizar la diferencia entre la simulación y la realidad, conocida en la literatura como "Reality Gap" [3].

Una de las dificultades principales del proyecto, fue la mecánica del robot: se tuvieron que rediseñar e imprimir varias veces algunas partes debido a la falta de experiencia en el campo y la intención inicial de abaratar costos, lo que generó varias versiones y demoras en el proyecto. De todas maneras, se cree que el resultado final llegó al mejor nivel posible con las herramientas y materiales disponibles en el mercado.

Para poder lograr los objetivos propuestos, durante el desarrollo del proyecto se tuvieron que abordar conceptos de aprendizaje por refuerzo (Soft Actor-Critic y su modificación implementando un enfoque multi-objetivo), sistemas de control (filtro de Kalman, modelizado de motores, control PID), mecánica (diseño de la estructura, implementación de sistemas articulados, distribución de pesos), robótica (cinemática inversa del robot) y herramientas de simulación, siendo implementados con éxito de manera individual y coordinada para garantizar el funcionamiento del robot. Fue una buena oportunidad para familiarizarse con los distintos conceptos y disciplinas requeridas por el proyecto, que a su vez integraron muchas de las asignaturas vistas durante la carrera.

Referencias

[1] A. D. Nimo, J. M. Piñeiro, J. Esarte, P. D. Folino, and S. Alberino, "Learning quadrupedal motion through multi-objective soft actor-critic," in *2024 IEEE Biennial Congress of Argentina (ARGENCON)*, 2024, pp. 1–8.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction, second edition*, ser. Adaptive Computation and Machine Learning. MIT Press, 2018.

[3] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, "Sim-to-real: Learning agile locomotion for quadruped robots," 2018.

[4] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine, "Soft actor-critic algorithms and applications," 2019.

[5] T. Schaul, D. Horgan, K. Gregor, and D. Silver, "Universal value function approximators," in *Proceedings of the 32nd International Conference on International Conference on Machine Learning*, ser. ICML'15, vol. 37. JMLR.org, 2015, p. 1312–1320.

[6] STMicroelectronics, "Nucleo-f412zg," https://www.st.com/en/evaluation-tools/nucleo-f412zg.html, 2025.

[7] S. Herzog, "Efficient dsp implementation of median filtering for real-time audio noise reduction," *Proc. of the 16th Int. Conference on Digital Audio Effects (DAFx-13)*, 2013.

[8] G. Welch and G. Bishop, "An introduction to the kalman filter," 1995.

[9] S. Studio, "Seed xiao ble sense nrf52840," https://www.seeedstudio.com/Seeed-XIAO-BLE-Sense-nRF52840-p-5253.html, 2025.

[10] R. Pi, "Raspberry pi 4 tech specs," https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/, 2025.

[11] B. Alp, "Diy quadruped robot," https://grabcad.com/library/diy-quadruped-robot-1, 2021.

[12] Cyberbotics, "Webots reference manual," https://cyberbotics.com/doc/reference/index, 2025.